

Σavante

**Lenguajes de Interrogación
de Bases de Datos. Estándar
ANSI SQL. Procedimientos
almacenados, eventos
y disparadores**

ÍNDICE

1. Lenguajes de interrogación de B.D.	6
2. Lenguaje SQL	9
2.1. Evolución de versiones	11
2.2. Clasificación del uso de SQL	13
2.2.1. Según el Momento de Compilación	13
2.2.2. Según el Modo de Ejecución	14
2.3. Tipos de datos. Dominios	14
2.3.1. Cadena de caracteres	15
2.3.2. Binario	17
2.3.3. Conversión de cadena a binario	18
2.3.4. Conversión de binario a otros tipos	18
2.3.5. Datos numéricos	18
2.3.5.1. Almacenar valores ENTEROS	19
2.3.5.2. Almacenar valores numéricos EXACTOS con decimales	20
2.3.5.3. Almacenar numéricos APROXIMADOS con decimales	20
2.3.5.4. Almacenar valores MONETARIOS	21
2.3.6. Datos tipo fecha y hora	21
2.3.7. Implementación de tipos de datos según el sistema gestor	23
2.4. Elementos de SQL	25
2.4.1. Sentencias SQL	26
2.4.2. Comandos SQL	28
2.4.3. Cláusulas	29
2.4.3.1. Predicados	31
2.4.4. Operadores	32
2.4.4.1. Operadores lógicos o booleanos	32
2.4.4.2. Operadores aritméticos	33
2.4.4.3. Operadores de comparación	34
2.4.4.3.1. Operador like	35
2.4.4.3.2. Operador IN	37

2.4.5. Funciones de agregado	37
2.4.6. Sintaxis SQL y su precedencia	38
2.5. Transacciones	39
3. ANSI SQL. Subconjuntos de lenguajes	40
4. DDL	41
4.1. Modificación de la estructura de los objetos de la B.D.	42
4.1.1. Bases de Datos	42
4.1.2. Tablas	44
4.1.3. Campos	46
4.1.3.1. Alter table	47
4.1.3.2. Campos autoincrementales	49
4.1.3.3. Definir campos como índices	51
4.1.3.4. Restricciones a los campos de las tablas	54
4.2. Relaciones entre las tablas: FOREIGN KEY y REFERENCES	57
4.3. Definición de vistas CREATE VIEW	59
5. DML	62
5.1. AS (alias)	64
5.2. Consultas de selección	66
5.2.1. Consultas básicas	67
5.2.1.1. Seleccionar campos (SELECT con cláusula FROM)	68
5.2.1.1.1. DISTINCT	70
5.2.1.1.2. ORDER BY	72
5.2.1.1.3. Cálculos sobre atributos	73
5.2.1.2. Indicar requisitos (Cláusula WHERE)	74
5.2.1.2.1. Sintaxis y ejemplos	76
5.2.2. Subconsultas	81
5.2.2.1. Ejemplos de Subconsultas	82
5.2.2.2. Agrupamiento de registros: Cláusulas GROUP BY y HAVING	88
5.2.2.2.1. Cláusula GROUP BY	90
5.2.2.2.2. Cláusula HAVING	93

5.2.3. Consultas de UNION	94
5.2.4. Combinación entre tablas (JOIN)	99
5.2.4.1. Combinación interna INNER JOIN	101
5.2.4.2. Combinación externa OUTER JOIN	103
5.2.4.2.1. LEFT JOIN	103
5.2.4.2.2. RIGHT JOIN (o RIGHT OUTER JOIN)	105
5.2.4.2.3. FULL JOIN	106
5.2.4.3. Combinación cruzada CROSS JOIN	109
5.3. Consultas de acción	109
5.3.1. INSERT	110
5.3.2. UPDATE	110
5.3.3. DELETE	111
5.3.4. MERGE	112
6. DCL	112
6.1. Comandos DCL	112
6.1.1. GRANT	113
6.1.2. REVOKE	114
7. Otras subclasificaciones de SQL	115
7.1. TCL	115
7.1.1. COMMIT	117
7.1.2. ROLLBACK	117
7.1.3. SAVEPOINT	118
7.1.4. RELEASE SAVEPOINT	119
7.1.5. SET TRANSACTION	119
7.2. CCL (Control de Cursores)	120
7.2.1. Creación de un cursor	120
7.2.2. Operaciones	121
7.2.3. Utilizar un cursor para manipular filas	122
7.2.3.1. FETCH	123
7.2.4. Monitorizar un cursor	124

8. Procedimientos almacenados	125
8.1. Palabras clave	127
8.2. Eventos y disparadores (<i>Triggers</i>)	130
8.2.1. Crear o eliminar un TRIGGER	132
8.2.1.1. Sintaxis de CREATE TRIGGER	133
8.2.1.2. Sintaxis de DROP TRIGGER	135
8.2.2. Extensiones OLD y NEW de MySQL para los disparadores	135
8.2.3. Gestión de errores	136
8.3. Snapshots en SQL	137
9. Bibliografía	138



1. Lenguajes de interrogación de B.D.



Figura 1. Fuente: Pixabay



Recuerda ver clases emitidas en temario audiovisual

Las clases impartidas en directo y disponibles en Campus, en Temario Audiovisual, te ayudarán al entendimiento de la unidad.

[ACCEDE DIRECTAMENTE DESDE AQUÍ](#)

QL, siglas de Query Language, es un lenguaje especializado para definir consultas y operaciones sobre bases de datos.

Se traduce al español como lenguajes de consulta (o lenguajes de interrogación).

Las consultas se realizan con comandos de programación estructurados.

Por tanto, podemos decir que un lenguaje de interrogación es el que se utiliza para tratar con los SGBD (Sistema Gestor de Base de Datos).



+ Info

En origen se crearon únicamente para consultar bases de datos, pero enseguida aparecieron instrucciones para poder modificar los datos.

Todos los SGBD modernos incorporan lenguajes de consulta que permiten consultar y modificar los datos.

Los lenguajes de consulta operan a un mayor nivel de abstracción que los lenguajes de programación genera y pueden ser clasificados de acuerdo a si son:

- **Lenguajes procedimentales.**

(Pueden verse con el nombre de "procedurales", por una mala traducción al español).

El usuario indica órdenes al sistema para que realice los procesos necesarios para obtener la información que el usuario solicita.

El Álgebra relacional es un lenguaje procedimental (ya estudiado en la unidad 5 "Sistemas de Gestión de Bases de Datos Relacionales, No Relacionales y Orientadas a Objetos", del Bloque II Tecnología Básica.

- **Lenguajes NO procedimentales, llamados Declarativos.**

(Pueden verse con el nombre de "no procedurales", por una mala traducción al español).

El usuario describe la información que desea obtener de la base de datos, pero no indica un procedimiento concreto que se llevará a cabo para obtener esa información.

El Cálculo relacional es un lenguaje de consulta, de tipo declarativo, que describe la respuesta deseada sobre una base de datos sin especificar cómo obtenerla. Tiene dos variantes:

- El cálculo relacional de tuplas (**TRC**).

Las variables son **tuplas**.

- El cálculo relacional de dominios (**DRC**).

Las variables son **atributos de las tuplas**.



+ Info

El Álgebra y el Cálculo relacional son lenguajes puros (ya que están formados por instrucciones muy elementales, símbolos...) o formales de Bases de Datos, son métodos que logran los mismos resultados, pero la diferencia es:

- Álgebra relacional: tipo procedimental.
- Cálculo relacional: tipo declarativo, no procedimental.

El álgebra relacional y el cálculo relacional son lenguajes formales de base matemática. SQL, aunque está estandarizado por ISO, no es un lenguaje formal puro.

No todos los lenguajes de consulta son declarativos; **existen lenguajes de consulta procedimentales y declarativos**, siendo SQL el más representativo de estos últimos.

Existen numerosos lenguajes de consulta, muchos de ellos especializados según el tipo de datos o sistema:

- **QUEL**: lenguaje de acceso a bases de datos relacionales, muy similar a SQL. Basado en el cálculo relacional de tuplas.
- **Common Query Language (CQL)**: lenguaje formal para representar consultas para sistemas de recuperación de información como índices web o catálogos bibliográficos.
- **Query by Example (QBE)**: basado en el cálculo relacional de dominios. Es un lenguaje de consulta de bases de datos relacionales similar al lenguaje de consulta estructurado (SQL).
- **D**: lenguaje de consulta para sistemas de administración de bases de datos verdaderamente relacionales (truly relational database management systems - TRDBMS).
- **DMX**: lenguaje para modelos de minería de datos.
- **Datalog**: lenguaje de consulta para bases de datos deductivas.
- **ERROL**: lenguaje de consulta sobre el modelo entidad-relación (ERM), especialmente diseñado para bases de datos relacionales.
- **Gellish English**: lenguaje que puede ser usado para consultas en bases de datos Gellish English, para diálogos (pedidos y respuestas) como también para modelado de información y modelado de conocimiento.



- **ISBL:** lenguaje de consulta para PRTV, uno de los más recientes sistemas de administración de bases de datos.
- **MQL:** lenguaje de consulta de quimio informática para búsqueda de subestructuras permitiendo propiedades nominales y numéricas.
- **MDX:** lenguaje de consulta para bases de datos OLAP.
- **OQL:** es un lenguaje de consulta orientado a objetos utilizado en bases de datos orientadas a objetos. Fue desarrollado como parte del estándar ODM (Object Data Management) por el Object Data Management Group (ODMG) y tiene una sintaxis similar a SQL, pero adaptada para trabajar con objetos en lugar de solo con datos relacionales.
- **OCL:** (Object Constraint Language - lenguaje de restricciones de objetos). Pese a su nombre, OCL es también un lenguaje de consulta de objetos y un estándar OMG.
- **OPath:** pensado para el uso consultando almacenes WinFS.
- **Poliqarp Query Language:** es un lenguaje de consulta especial diseñado para analizar texto con anotaciones. Usado en el motor de búsqueda Poliarp.
- **SMARTS:** estándar de quimio informática para búsqueda de subestructuras.
- **SPARQL:** lenguaje de consulta para grafos RDF.
- **SuprTool:** lenguaje de consulta propietario para SuprTool,6 un programa de acceso a bases de datos para obtener datos en Image/SQL (TurboIMAGE) y bases de datos Oracle.
- **TMQL Topic Magic Queen Lion:** lenguaje de consulta para Topic Maps.
- **XQuery:** lenguaje de consulta para fuentes de datos XML.
- **EPL:** lenguaje de consulta para el procesamiento de eventos complejos en tiempo real.

2. Lenguaje SQL

SQL, por sus siglas en inglés Structured Query Language (en español, lenguaje de consulta estructurado).

Es un lenguaje declarativo, (se dice qué se quiere conseguir, pero no se especifica cómo) diseñado para administrar, y recuperar información de bases de datos relacionales.

Aunque SQL, permite modificar bases de datos, su origen era para realizar consultas.

Las consultas se realizan con comandos estructurados y normalizados.



SQL está estandarizado por ISO y es el estándar de facto para bases de datos relacionales, aunque no es un lenguaje formal puro desde el punto de vista matemático.



+ Info

Estándar de facto es aquel patrón o norma que se caracteriza por no haber sido consensuada ni legitimada por un organismo de estandarización al efecto.

Por el contrario, se trata de una norma generalmente aceptada y ampliamente utilizada por iniciativa propia de un gran número de interesados.

Se dice que SQL se basa en el álgebra y el cálculo relacional, y que las operaciones relacionales se expresan mediante cláusulas específicas.

Por tanto, aunque se haga referencia al lenguaje SQL como "lenguaje de consultas", (ya que este fue su origen), puede hacer mucho más, como:

- Definir la estructura de los datos: **Definición.**
- Modificación de los datos: **Manipulación.**

Inserción de datos, consultas, actualizaciones y borrado, la creación y modificación de esquemas (vistas).

- Especificar restricciones de seguridad: **Control de Datos.**

Como describió Edgar Frank Codd en su artículo de investigación 1970, SQL fue uno de los primeros lenguajes comerciales para el modelo relacional, para grandes bancos de datos compartidos. Aunque no se adhería totalmente al modelo relacional descrito por Codd, se convirtió en el lenguaje de base de datos más usado.

Historia

A mediados de la década de 1970, los laboratorios de IBM, (San José Research Laboratory), basándose en el modelo relacional propuesto por Edgar Frank Codd, crearon el lenguaje SEQUEL (Structured English QUery Language), y posteriormente IBM lo implementó en 1977 para poder gestionar los datos almacenados en el nuevo software de base de datos llamado SYSTEM R.



SQL fue una versión evolucionada de SEQUEL, y se convirtió en el lenguaje por excelencia de los SGBD de la época.

En 1979, se lanzó la primera versión denominada Oracle V, por la compañía "Relational Software", que supo ver el gran potencial del lenguaje SQL, compañía que luego se convirtió en Oracle.

En octubre de 1986, el ANSI adoptó SQL como lenguaje estándar para la gestión de bases de datos relacionales. Posteriormente, en 1987, la Organización Internacional de Normalización (ISO, International Organization for Standardization) lo ratificó como estándar internacional bajo la denominación SQL/86. A partir de entonces, el estándar SQL ha sido objeto de sucesivas revisiones, dando lugar a versiones como SQL/89, SQL/92 y posteriores, hasta llegar a SQL/2008.



Imprescindible

El Instituto Nacional Estadounidense de Estándares, más conocido como **ANSI** (por sus siglas en inglés: American National Standards Institute), es una organización sin fines de lucro, fundada en 1966, que supervisa el desarrollo de estándares para productos, servicios, procesos y servicios en los Estados Unidos.

Desde entonces, el estándar SQL ha sido revisado para incluir más características.

A pesar de la existencia de ambos estándares, la mayoría de los códigos SQL no son completamente portables entre diferentes sistemas de bases de datos sin que sea necesario realizar otros ajustes.

2.1. Evolución de versiones

A lo largo de los años se han ido revisando las versiones de SQL, en ocasiones con mínimas modificaciones, y en otros casos con ampliaciones importantes de los usos de SQL.

- **1986: SQL/86 (Alias SQL/87)**

Fue la primera versión del estándar SQL adoptada por ANSI en 1986 y ratificada por ISO en 1987.

- **1989: SQL/89**

Se realiza una mínima revisión de la versión anterior.



- **1992: SQL-92 (Alias SQL2)**

Se realiza una revisión a mayor nivel.

- **1999: SQL/1999 (Alias SQL2000)**

También conocido como SQL3, fue una revisión importante del estándar SQL en la que se incorporaron nuevas funcionalidades, entre las que destacan:

- Expresiones regulares.
- Consultas recursivas.
- Triggers.
- Características orientadas a objetos.

Algunas de las nuevas características, requirieron aclaraciones en el posterior SQL:2003.

- **2003: SQL/2003**

Nuevamente, se introducen nuevas características:

- Características de XML.
- Cambios en las funciones.
- Estandarización del objeto SEQUENCE y columnas con campos autonuméricos.

- **2006: SQL/2006**

Se definen las formas en que SQL se puede utilizar conjuntamente con XML en el estándar ISO/IEC 9075-14:2006, como, por ejemplo:

- Formas para que en una B.D. SQL se pueda importar y guardar datos XML, manipulándolos en la B.D. y pudiendo publicar los datos convencionales SQL en forma XML.
- Se añaden facilidades para que las aplicaciones integren dentro de su código SQL el uso de XQuery.

- **2008: SQL/2008**

Se consolidan y estandarizan diversas características del lenguaje, como:

- La sentencia TRUNCATE.
- El uso de ORDER BY fuera de cursores.
- Los disparadores de tipo INSTEAD OF.



- **2011: SQL/2011**

Se añaden características de manejo de datos temporales, y tablas versionadas en el sistema.

- **2016: SQL/2016**

Se añaden:

- Coincidencia de patrones por filas.
- Funciones de tabla.
- JSON.

- **2019: SQL/2019**

Se amplía el soporte para tipos de datos complejos, como los arrays multidimensionales.

Actualmente, los comités ANSI e ISO continúan en un proceso permanente de revisión y evolución del lenguaje SQL, con el objetivo de incorporar nuevas características acordes con la evolución de los sistemas de bases de datos y de los tipos de datos, como el tratamiento de datos multimedia. Estas revisiones buscan ampliar las capacidades del lenguaje y reforzar su uso como lenguaje estándar e independiente para la gestión de bases de datos, frente a su utilización integrada o embebida dentro de otros lenguajes de programación.

2.2. Clasificación del uso de SQL

SQL puede clasificarse de diferentes formas:

- Según el **Momento de Compilación**.
- Según el **Modo de Ejecución**.

2.2.1. Según el Momento de Compilación

SQL Estático

Las sentencias SQL están definidas previamente en el código fuente y se "compilan" o preparan antes de ejecutarse.

Esto proporciona mayor eficiencia, ya que el plan de ejecución puede reutilizarse.



SQL Dinámico

Son sentencias SQL que se construyen y se compilan en tiempo de ejecución.

Permite, que los usuarios ingresen sus propias consultas SQL, haciendo que sea muy flexible, pero las sentencias son menos eficientes, que en el SQL estático.

2.2.2. Según el Modo de Ejecución

Interactivo o programático

El modo de ejecución puede ser **Interactivo** (desde una consola) o **Programático** (desde un programa).

El modo programático

Se implementa de dos formas:

- **SQL Embebido/Incrustado:** Mezclando sentencias SQL con el código de un lenguaje host.
- **SQL Modular:** la lógica SQL reside en procedimientos almacenados, funciones o vistas; el programa solo los llama por nombre.

Estos conceptos son independientes: Cualquier combinación es posible. Por ejemplo, puedes tener SQL Interactivo Estático, SQL Embebido Dinámico o SQL Modular Estático, etc.

2.3. Tipos de datos. Dominios

En SQL, podemos trabajar con diferentes tipos de datos, los cuales tienen sus propias características y, por tanto, solo podrán albergar un determinado tipo de valor (dominio).

Los dominios son reglas que se aplican a un campo en una tabla, restringen el conjunto de valores permitidos para ese campo y, por tanto, aplican la integridad de datos.

Aplicar la integridad de datos significa que solo se permite que se introduzcan en el campo los valores especificados.



Recuerda

Dominio.

Describe un conjunto de posibles valores para cierto atributo.



Puesto que restringe los valores del atributo, puede ser considerado como una restricción.

Matemáticamente, atribuir un dominio a un atributo significa "cualquier valor de este atributo debe ser elemento del conjunto especificado".

Como sabemos existen distintos tipos de Sistemas Gestores de Bases de Datos que utilizan SQL como lenguaje principal. Si bien en esta unidad la idea es hablar de las características comunes del lenguaje, en el epígrafe "Implementación de tipos de datos según el Sistema Gestor" matizamos compatibilidades de los tipos de datos con los distintos SGBDs.

2.3.1. Cadena de caracteres

Las cadenas de caracteres son valores literales utilizados para almacenar texto en una base de datos. Pueden contener letras, números, símbolos y espacios, y su uso es fundamental para representar información como nombres, direcciones o descripciones. Dependiendo del sistema gestor de bases de datos (SGBD), pueden ser de longitud fija o variable, y utilizar distintas codificaciones de caracteres.

Codificación

El tamaño en bytes que ocupa una cadena depende de la codificación de caracteres.

Antiguamente se usaban codificaciones limitadas como ASCII o Latin-1, capaces de representar solo una parte del conjunto de caracteres posibles.

Actualmente, la mayoría de los sistemas emplean Unicode, un estándar que permite representar caracteres de casi todos los idiomas.

Las formas de codificación Unicode más comunes son:

- **UTF-8** (Formato de Transformación Unicode de 8 bits): es la codificación más extendida, eficiente en almacenamiento y compatible con ASCII.
- **UTF-16** (Formato de Transformación Unicode de 16 bits): utiliza una o dos unidades de 16 bits por carácter; es habitual en sistemas como SQL Server y Oracle para los tipos Unicode.



Tipos principales de datos de texto

Los tipos de datos de texto más comunes en SQL se clasifican según su longitud y codificación. El parámetro *n* define el número máximo de caracteres que puede almacenar el campo, y el espacio ocupado en memoria dependerá de la codificación.

- **CHAR(*n*):** Presente en todos los sistemas SQL (estándar ANSI SQL). Almacena una cadena de longitud fija. Si el valor tiene menos caracteres que *n*, se rellena con espacios hasta alcanzar la longitud indicada.
- **VARCHAR(*n*):** Utilizado en la mayoría de SGBD (MySQL, PostgreSQL, Oracle, SQL Server). Almacena una cadena de longitud variable. Solo ocupa el espacio necesario para los caracteres almacenados, más una pequeña cantidad de metadatos.
 - **En SQL Server**, el límite estándar es 8000 bytes, pudiendo ampliarse con VARCHAR(MAX) hasta 2 GB.
 - **En Oracle**, VARCHAR2(*n*) cumple la misma función, con límites dependientes de la versión.
 - **En PostgreSQL y MySQL**, VARCHAR(*n*) puede alcanzar hasta 65.535 bytes según la configuración de la tabla y la codificación.
- **NCHAR(*n*) y NVARCHAR(*n*):** Tipos diseñados para almacenar texto en Unicode (normalmente UTF-16).

Son específicos de SQL Server y Oracle, aunque MySQL y PostgreSQL pueden manejar Unicode directamente en CHAR y VARCHAR si la base de datos está configurada con una codificación Unicode (como UTF-8).

- **NCHAR(*n*)** tiene longitud fija.
- **NVARCHAR(*n*)** tiene longitud variable. En SQL Server puede declararse como NVARCHAR(MAX) para textos muy largos.
- **TEXT o CLOB** (Character Large Object): Tipos diseñados para almacenar grandes volúmenes de texto, como documentos o párrafos extensos. Su implementación y restricciones (por ejemplo, en la indexación o comparación) varían entre sistemas gestores.
 - **En SQL Server**, TEXT y NTEXT están en desuso; se recomienda usar VARCHAR(MAX) o NVARCHAR(MAX).
 - **En Oracle**, el equivalente es CLOB (texto no Unicode) y NCLOB (texto Unicode).
 - **En MySQL**, VARCHAR(*n*) está limitado por el tamaño máximo de fila (hasta 65.535 bytes, dependiendo de la codificación y del resto de columnas).
 - **En PostgreSQL**, VARCHAR(*n*) y TEXT pueden almacenar cadenas de hasta aproximadamente 1 GB.



2.3.2. Binario

Los tipos de datos binarios permiten almacenar información no textual, como imágenes, documentos, claves cifradas o archivos ejecutables. Su tamaño puede ser de longitud fija o variable, y depende del parámetro *n* especificado al definir la columna o variable.

Cuando no se especifica *n* en una instrucción de definición de datos o declaración de variables, la longitud predeterminada suele ser 1.

Si no se especifica *n* al usar funciones de conversión como CAST o CONVERT, algunos sistemas adoptan un valor por defecto, generalmente 30 bytes, aunque este comportamiento puede variar según el SGBD.

- **BINARY(*n*):** Almacena datos binarios de longitud fija. El valor *n* define la longitud en bytes, normalmente entre 1 y 8000. El tamaño de almacenamiento es exactamente *n* bytes. Se recomienda su uso cuando los datos almacenados tienen tamaños uniformes (por ejemplo, códigos hash o identificadores cifrados).
- **VARBINARY(*n*):** Almacena datos binarios de longitud variable. El valor *n* puede oscilar entre 1 y 8000 bytes, y también puede usarse la palabra clave MAX para admitir longitudes mayores (hasta $2^{31}-1$ bytes, es decir, 2 GB).

Se recomienda utilizarlo cuando el tamaño de los datos varía considerablemente o cuando los valores pueden superar los 8000 bytes.

El tamaño de almacenamiento corresponde a la longitud real de los datos más un pequeño espacio adicional (normalmente 2 bytes para la gestión interna).

El estándar ANSI SQL define como sinónimo de VARBINARY el término BINARY VARYING.

- **BLOB** (Binary Large Object): Tipo estándar para almacenar grandes volúmenes de datos binarios (imágenes, audio, vídeo, etc.).
 - **En MySQL**, existen variantes (TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB) con diferentes tamaños máximos.
 - **En Oracle**, el tipo BLOB cumple la misma función.
 - **En PostgreSQL**, no existe un tipo BLOB directo, pero se utiliza BYTEA o almacenamiento mediante objetos grandes (large objects) gestionados con OID.
 - **En SQL Server**, el tipo equivalente es VARBINARY(MAX) o el antiguo IMAGE (ya obsoleto).

En SQL es posible convertir valores entre tipos de datos de texto (string) y tipos binarios (binary o varbinary).

Estas conversiones deben realizarse con precaución, ya que pueden producir pérdida o alteración de información dependiendo de la longitud y del sistema gestor utilizado.



2.3.3. Conversión de cadena a binario

Cuando se convierten datos de tipo CHAR, VARCHAR, NCHAR, NVARCHAR, TEXT o NTEXT a un tipo BINARY o VARBINARY de diferente longitud, los datos pueden truncarse o rellenarse.

En SQL Server, si la longitud del destino es inferior a la del valor de origen, los datos se truncan por la derecha. Si es mayor, se rellenan con ceros hexadecimales (0x00) hasta alcanzar la longitud definida.

Este comportamiento puede ser útil cuando se requiere mover datos en formato binario o realizar operaciones que necesiten un tipo no textual.

Si un valor se convierte a binario con un tamaño suficientemente grande y luego se vuelve a convertir a su tipo original, el resultado será el mismo solo si ambas conversiones se ejecutan en la misma versión del sistema gestor. Entre versiones o SGBD distintos, los resultados pueden variar por diferencias en codificación y representación interna.

2.3.4. Conversión de binario a otros tipos

Cuando se convierten datos de tipo BINARY o VARBINARY a otros tipos (por ejemplo, texto o enteros), los valores también pueden truncarse o rellenarse, pero en este caso el relleno o truncamiento se realiza por la izquierda.

En SQL Server, es posible convertir tipos numéricos como INT, SMALLINT o TINYINT a BINARY o VARBINARY.

Sin embargo, si durante la conversión se produce truncamiento, al convertir de nuevo el valor binario a un entero se obtendrá un resultado diferente al valor original.

2.3.5. Datos numéricos

Se pueden almacenar diferentes tipos de datos numéricos, y hay que tener en cuenta que no se permite ingresar un valor fuera de rango.

Si ingresamos una cadena, en un tipo de dato numérico, el sistema (según cómo esté definido), intentará convertirla a valor numérico. Si dicha cadena consta solamente de dígitos, la conversión se realiza internamente, y a continuación se verifica si el valor está dentro del rango, si es así, se ingresa en la tabla.

Si no está dentro del rango, el comportamiento dependerá del sistema de gestión de bases de datos (SGBD): algunos SGBD mostrarán un mensaje de error y no ejecutarán la sentencia, mientras que otros pueden emitir un aviso y almacenar el valor ajustado al máximo permitido para ese atributo.

Si la cadena contiene caracteres que SQL no puede convertir a valor numérico, muestra un mensaje de error y la sentencia no se ejecuta.



+ Info

Por ejemplo, si se define un campo de tipo decimal(5,2), máximo de 5 dígitos contando con un tope de dos decimales, e ingresamos la cadena '12.22', se convierte al valor numérico 12.22 y se ingresa.

Si ingresamos la cadena '1234.56', la convierte al valor numérico 1234.56, pero al ser el valor máximo permitido 999.99 mostrará un mensaje indicando que está fuera de rango.

Si ingresamos el valor '12 y 25', SQL no puede realizar la conversión y muestra un mensaje de error.

Es importante elegir el tipo de dato adecuado según el caso, el más preciso. Por ejemplo, si un campo numérico almacenará valores positivos menores a 255, el tipo "int" no es el más adecuado, conviene el tipo "tinyint", de esta manera usamos el menor espacio de almacenamiento posible.

Si vamos a guardar valores monetarios menores a 200000 conviene emplear SMALLMONEY en lugar de MONEY (en sistemas como SQL Server), aunque en entornos estándar se recomienda el uso de DECIMAL o NUMERIC por su mayor precisión y portabilidad.

Podemos distinguir los siguientes tipos de datos numéricos:

- Valores enteros.
- Valores numéricos EXACTOS con decimales.
- Valores numéricos APROXIMADOS con decimales.
- Valores monetarios.

2.3.5.1. Almacenar valores ENTEROS

- SMALLINT, INTEGER o INT, BIGINT: Permiten representar números enteros con distintos rangos, según el tamaño de almacenamiento requerido.
 - **SMALLINT**: Es un entero corto, puede contener hasta 5 dígitos. Su rango aproximado va desde -32.768 hasta 32.767. Longitud: 2 bytes.
 - **INTEGER o INT**: Representa un número entero estándar. Su tamaño habitual es de 4 bytes, y su rango aproximado es de -2.147.483.648 a 2.147.483.647.
 - **BIGINT**: Permite almacenar valores enteros de gran magnitud. Su rango aproximado es de -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807. Longitud: 8 bytes.



Algunos sistemas gestores de bases de datos permiten trabajar con rangos extendidos mediante tipos específicos o configuraciones personalizadas.

- **TINYINT**: Disponible en algunos SGBD para representar valores muy pequeños. Puede almacenar valores enteros comprendidos entre 0 y 255. Longitud: 1 byte.
- **BIT**: Tipo que representa valores booleanos (0 o 1). Aunque conceptualmente ocupa 1 bit, en muchos sistemas se almacena en 1 byte. No todos los SGBD lo implementan como tipo nativo.

2.3.5.2. Almacenar valores numéricos EXACTOS con decimales

Se utiliza para almacenar valores numéricos EXACTOS con decimales, especificando la cantidad de cifras a la izquierda y derecha del separador decimal. Hablamos aquí de precisión fija o exacta.

DECIMAL(p, s) o NUMERIC(p, s):

Representan valores numéricos con precisión fija.

La precisión (p) indica el número total de dígitos que puede tener el número (tanto a la izquierda como a la derecha del punto decimal), mientras que la escala (s) indica cuántas cifras se reservan para la parte decimal.

Ambos tipos son equivalentes según el estándar SQL, aunque algunos sistemas gestores de bases de datos pueden implementar diferencias internas mínimas entre ellos o usar solo uno de los dos.

El rango máximo de precisión depende del SGBD; por ejemplo, muchos permiten hasta 38 dígitos de precisión total, aunque otros admiten configuraciones superiores mediante parámetros o extensiones.

Estos tipos se utilizan habitualmente en cálculos financieros, científicos o contables, donde la exactitud del valor es prioritaria frente a la velocidad de cálculo.

2.3.5.3. Almacenar numéricos APROXIMADOS con decimales

Para almacenar valores numéricos aproximados con decimales se utiliza la precisión flotante.

Este tipo de precisión se relaciona con la cantidad de dígitos significativos que pueden almacenarse y emplearse en los cálculos, y es ideal para aplicaciones científicas, gráficas o de ingeniería, donde se requiere representar un amplio rango de valores más que una exactitud absoluta.

Los valores de precisión flotante se almacenan en formato binario, lo que puede provocar pequeños errores de redondeo al representar ciertos valores decimales.

- **FLOAT, REAL y DOUBLE PRECISION** son los principales tipos de datos de precisión flotante.
- **FLOAT**: adecuado para aplicaciones donde la rapidez es esencial, pero la precisión no es crítica. Suele ofrecer alrededor de 7 dígitos decimales de precisión y ocupar 4 bytes de almacenamiento.



- **DOUBLE o DOUBLE PRECISION:** almacena valores con mayor precisión que FLOAT. Suele ofrecer entre 15 y 16 dígitos decimales y ocupar 8 bytes de almacenamiento.
- **REAL:** aunque es un tipo de dato válido en SQL, su uso varía según el sistema gestor de bases de datos. En algunos SGBD puede ser sinónimo de DOUBLE (como en ciertas configuraciones de MySQL) o equivalente a FLOAT (como en PostgreSQL).

En algunos sistemas, el tipo DOUBLE se reconoce como sinónimo de DOUBLE PRECISION, aunque su disponibilidad o comportamiento exacto puede depender del motor de base de datos.

Estos tipos se utilizan cuando se necesita representar valores grandes o muy pequeños con una precisión relativa suficiente, aceptando un margen de error mínimo en los cálculos.

2.3.5.4. Almacenar valores MONETARIOS

Para almacenar valores monetarios se emplean tipos de datos diseñados para representar cantidades económicas con decimales fijos y un control exacto de la precisión.

MONEY, SMALLMONEY: Utilizados para representar valores monetarios con decimales fijos.

No forman parte del estándar SQL ANSI, pero están presentes en varios sistemas gestores de bases de datos comerciales, especialmente en SQL Server.

En algunos sistemas pueden existir variantes o configuraciones internas que permiten trabajar con cantidades de mayor tamaño.

- **MONEY:** Puede tener hasta 19 dígitos en total, de los cuales 4 corresponden a decimales. Su rango aproximado es de $-922.337.203.685.477,5808$ a $922.337.203.685.477,5807$.
- **SMALLMONEY:** Acepta valores comprendidos aproximadamente entre $-214.748,3648$ y $214.748,3647$.

Estos tipos son adecuados para cálculos financieros o presupuestarios, aunque muchos entornos profesionales prefieren el uso de DECIMAL o NUMERIC con una precisión y escala definidas, ya que son tipos estandarizados y más portables entre diferentes sistemas gestores.

2.3.6. Datos tipo fecha y hora

Los tipos de datos fecha y hora permiten almacenar información temporal, ya sea solo la fecha, solo la hora, o una combinación de ambas.

Estos tipos resultan fundamentales para registrar eventos, programar operaciones o calcular intervalos entre momentos específicos.

- **DATE:** Almacena únicamente una fecha (año, mes y día). Forma parte del estándar SQL y está disponible en la mayoría de los SGBD (SQL Server, Oracle, PostgreSQL y MySQL). Su rango y precisión pueden variar según el sistema gestor.



- **TIME:** Almacena solo la hora del día, incluyendo horas, minutos, segundos y fracciones de segundo.

También es parte del estándar SQL y está implementado en los principales SGBD, aunque con ligeras diferencias de precisión.

La longitud suele variar entre 6 y 8 bytes, dependiendo del nivel de precisión (de 0 a 7 dígitos fraccionarios).

Rango habitual: desde 00:00:00.0000000 hasta 23:59:59.9999999.

- **DATETIME** y variantes: Combinan fecha y hora en un solo valor. Pueden incluir fracciones de segundo y su comportamiento puede variar entre sistemas. Algunos SGBD ofrecen versiones mejoradas con mayor precisión o rango ampliado:

- **DATETIME** (SQL Server, MySQL): Longitud de 8 bytes. Precisión de hasta 3,33 milisegundos, redondeando a incrementos de .000, .003 o .007 segundos.

Rango (SQL Server): desde el 1 de enero de 1753 hasta el 31 de diciembre de 9999.

- **DATETIME2** (SQL Server): Longitud entre 6 y 8 bytes, según la precisión especificada. Mejora datetime ampliando su rango (del 1 de enero del año 1 al 31 de diciembre de 9999) y permitiendo ajustar la precisión fraccional (de 0 a 7 dígitos).
- **SMALLDATETIME** (SQL Server): Longitud de 8 bytes. Almacena fecha y hora con menor precisión, redondeando al minuto más cercano.

Rango: desde el 1 de enero de 1900 hasta el 6 de junio de 2079.

Precisión: un minuto (ejemplo: 2023-10-27 15:23:00).

- **TIMESTAMP** (MySQL, PostgreSQL, estándar SQL): Similar a datetime, aunque su semántica varía. En MySQL almacena automáticamente la fecha y hora de inserción o actualización de la fila, mientras que en PostgreSQL y en el estándar SQL representa una marca de tiempo simple con o sin zona horaria.
- **DATETIMEOFFSET / TIMESTAMP WITH TIME ZONE:** Tipos que combinan fecha y hora incluyendo además el desplazamiento de zona horaria (offset).
 - En **SQL Server**, DATETIMEOFFSET ocupa 10 bytes (con precisión predeterminada de 7 fraccionarios) y permite registrar la hora junto a su zona horaria (por ejemplo, -05:00 para EST)
 - En **PostgreSQL y Oracle**, se emplea el tipo TIMESTAMP WITH TIME ZONE con un propósito equivalente. Son ideales para aplicaciones globales que requieren almacenar tanto la hora local como la referencia universal (UTC).



- **TIMESTAMP / ROWVERSION:** Este tipo de dato no almacena una fecha ni una hora, aunque su nombre pueda inducir a error.

Representa un número binario de 8 bytes que cambia automáticamente cada vez que se inserta o modifica una fila.

Su finalidad es el control de concurrencia optimista, detectando si una fila ha sido modificada desde su última lectura.

El tipo **TIMESTAMP** está en desuso; se recomienda emplear **ROWVERSION** para mayor claridad.

- **INTERVAL:** Utilizado para representar duraciones o intervalos de tiempo.

Es parte del estándar SQL y está implementado en sistemas como PostgreSQL y Oracle, aunque no en SQL Server ni MySQL.

Cada sistema puede presentar diferencias en cuanto a rango de valores, precisión, almacenamiento y funciones asociadas. Es recomendable consultar la documentación del SGBD utilizado para conocer las características concretas de cada tipo.

2.3.7. Implementación de tipos de datos según el sistema gestor

El lenguaje SQL está regulado por un estándar ANSI/ISO que define un conjunto básico de tipos de datos y su comportamiento general. Este estándar es la base sobre la que se construyen los distintos sistemas gestores de bases de datos (SGBD). Sin embargo, cada gestor puede implementar estos tipos con variaciones en su nombre, tamaño máximo, precisión, reglas internas o funcionalidades extendidas.

A continuación, se resumen las características más relevantes en la implementación de tipos de datos en algunos de los SGBD más utilizados:

MySQL y MariaDB

MySQL y MariaDB comparten una implementación muy similar de los tipos de datos, ya que MariaDB surgió como una bifurcación de MySQL. Ambos gestores distinguen entre varios tipos de datos textuales y binarios con gran flexibilidad. El tipo `VARCHAR(n)` puede almacenar hasta 65.535 bytes por fila, aunque el límite efectivo depende del conjunto de caracteres y de otros campos definidos en la tabla.

Los tipos `TEXT` y `BLOB` se presentan en versiones con distinto tamaño máximo (`TINY`, `MEDIUM`, `LONG`), útiles para manejar grandes cantidades de información. Ninguno de los dos utiliza tipos `NVARCHAR` o `NCHAR` como tales, el soporte Unicode se gestiona mediante la codificación del conjunto de caracteres (por ejemplo, `utf8mb4`). En cuanto a fechas, `DATETIME` y `TIMESTAMP` se comportan de forma similar, aunque `TIMESTAMP` está vinculado al huso horario del servidor, y su valor puede ajustarse automáticamente según la zona configurada.

MariaDB mantiene un alto grado de compatibilidad con MySQL, aunque en versiones recientes ha introducido optimizaciones y funciones propias. Para la mayoría de casos prácticos, los tipos de datos se comportan de manera equivalente en ambos sistemas.



PostgreSQL

PostgreSQL permite una definición muy flexible y semánticamente coherente de los tipos de datos. Los tipos VARCHAR, TEXT y CHARACTER VARYING se comportan de forma equivalente, y no hay un límite práctico para su longitud. Para datos binarios se utiliza BYTEA, y para cadenas de texto grandes TEXT es completamente funcional.

PostgreSQL también incorpora el tipo UUID como nativo, además de JSON y JSONB para estructuras semiestructuradas. En cuanto a fechas, distingue claramente entre TIMESTAMP y TIMESTAMP WITH TIME ZONE (timestampz), con soporte completo para operaciones sobre zonas horarias.

SQL Server

SQL Server proporciona versiones extendidas de varios tipos, como VARCHAR(MAX), NVARCHAR(MAX) y VARBINARY(MAX), que permiten almacenar hasta 2 GB de datos. El tipo DATETIME2 ofrece mayor precisión y un rango más amplio que el tipo DATETIME. Para datos de tipo GUID, incluye el tipo nativo UNIQUEIDENTIFIER. También dispone de tipos específicos para valores monetarios, como MONEY y SMALLMONEY, aunque se recomienda cautela en cálculos financieros debido a su precisión fija.

SQL Server distingue claramente entre tipos con y sin soporte Unicode (VARCHAR frente a NVARCHAR) y admite la definición explícita de collations para la ordenación y comparación de cadenas.

Oracle

Oracle utiliza VARCHAR2 en lugar de VARCHAR por razones históricas, con tamaños máximos configurables (hasta 32.767 caracteres si se habilita MAX_STRING_SIZE=EXTENDED). Para grandes volúmenes de texto o binarios se emplean CLOB y BLOB. El tipo NUMBER permite una definición numérica precisa con control de precisión y escala. Oracle incorpora también tipos como RAW para datos binarios y ROWID para identificadores internos.

Su tipo DATE incluye hora por defecto, y dispone de TIMESTAMP con o sin zona horaria (TIMESTAMP WITH TIME ZONE y TIMESTAMP WITH LOCAL TIME ZONE).

SQLite

SQLite utiliza un sistema de tipado dinámico basado en afinidades de tipo. Acepta los tipos generales INTEGER, REAL, TEXT, NUMERIC y BLOB, sin requerir tamaño ni precisión específicos. El motor puede almacenar prácticamente cualquier tipo de valor en cualquier columna, aplicando reglas de conversión internas según la declaración del tipo.

Por ello, SQLite ofrece una flexibilidad mayor a costa de una validación de tipos más laxa, lo que lo hace ideal para aplicaciones ligeras o embebidas, aunque menos estricto en entornos donde la integridad del tipo sea prioritaria.



2.4. Elementos de SQL

Como en cualquier lenguaje, se utilizan palabras clave y reservadas, es decir, propias de SQL y que, por tanto, no pueden emplearse como identificadores, por ejemplo, para nombrar una tabla o una columna.

El lenguaje SQL está compuesto por diferentes elementos que se combinan en las instrucciones para crear, consultar, actualizar y manipular bases de datos.

Estos elementos son:

Sentencias o instrucciones:

Son las órdenes que se envían al sistema gestor, como SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, o DROP.

Cada sentencia cumple una función específica dentro del lenguaje SQL.

Comandos:

Representan los grupos de sentencias según su finalidad, como por ejemplo:

- **DDL** (Data Definition Language): para definir estructuras (CREATE, ALTER, DROP).
- **DML** (Data Manipulation Language): para manipular datos (SELECT, INSERT, UPDATE, DELETE).
- **DCL** (Data Control Language): para permisos (GRANT, REVOKE).
- **TCL** (Transaction Control Language): para control de transacciones (COMMIT, ROLLBACK, SAVEPOINT).

Cláusulas:

Son las partes que componen una sentencia y que definen sus condiciones o restricciones, como FROM, WHERE, ORDER BY, GROUP BY o HAVING.

Operadores:

Son símbolos o palabras utilizadas para realizar operaciones dentro de las sentencias.

Pueden ser de varios tipos:

- Lógicos (booleanos): AND, OR, NOT
- Aritméticos: +, -, *, /
- De comparación: =, <>, <, >, <=, >=



Funciones de agregado:

Son operaciones predefinidas que trabajan sobre conjuntos de registros para obtener resultados resumidos, como SUM(), AVG(), COUNT(), MIN() y MAX().

2.4.1. Sentencias SQL

Las sentencias o instrucciones SQL están compuestas por palabras clave, identificadores, constantes y delimitadores, y permiten consultar, definir, modificar o eliminar datos y objetos dentro de una base de datos.

Cada sentencia SQL se construye combinando diferentes elementos que deben seguir un orden y una sintaxis específica.

Como cualquier lenguaje formal, SQL utiliza palabras reservadas que no pueden emplearse como nombres de tablas, columnas u otros objetos del sistema.

Una sentencia SQL típica está formada por los siguientes componentes:

Comando de SQL:

Todas las sentencias comienzan con un comando, que es una palabra predefinida con un significado propio, como SELECT, INSERT, UPDATE, DELETE, CREATE o DROP.

Estas palabras son reservadas, por lo que no pueden utilizarse como identificadores dentro de la base de datos.

Los comandos se agrupan en distintos sublenguajes de SQL (DDL, DML, DCL, TCL), que estudiaremos más adelante.

Identificadores:

Son los nombres de las tablas, columnas, vistas, índices o cualquier otro objeto definido en la base de datos.

Sirven para hacer referencia a los elementos sobre los que actúa una sentencia.

Los identificadores deben ser únicos dentro de su ámbito, es decir, no pueden existir dos tablas con el mismo nombre ni dos campos con el mismo nombre dentro de una misma tabla.

Constantes (valores literales o escalares):

Representan valores específicos de datos que se incluyen directamente en las sentencias.

El formato de una constante depende del tipo de datos al que pertenece (numérico, alfabético, binario, monetario, etc.).



A continuación se muestran los tipos más comunes:

- Constantes de cadena de caracteres: se escriben entre comillas simples (' ') e incluyen caracteres alfanuméricos o especiales como (!), (@) o (#).
- Constantes binarias: su representación depende del SGBD; en sistemas como SQL Server se utiliza el prefijo 0x seguido de valores hexadecimales (0xAC, 0x12EF, 0x69048AEFDD010E).
- Constantes de tipo BIT: se representan habitualmente con 0 o 1, sin comillas. El comportamiento ante otros valores depende del SGBD.
- Constantes de tipo datetime: se escriben entre comillas simples y pueden adoptar distintos formatos de fecha y hora ('December 6, 1974', '12/6/74', '11:47:24', '05:24 PM').
- Constantes enteras (INTEGER): números enteros sin comillas ni separadores decimales (1974, 7).
- Constantes decimales: números con separador decimal (2471.2034, 3.0).
- Constantes de tipo FLOAT o REAL: representadas en notación científica (101.5E5, 0.5E-2).
- Constantes monetarias (MONEY): valores numéricos con decimales fijos. El uso de símbolos de moneda depende del SGBD y no forma parte del estándar SQL.

Para indicar números positivos o negativos, se emplean los operadores unarios + o -. Si no se indica el signo, se asume positivo (+56789153, -3289457, +123456.57, -9876543.77, *en algunos SGBD como SQL Server pueden emplearse símbolos de moneda* -\$45.56, +\$42356.99).

- Constantes de tipo UNIQUEIDENTIFIER (GUID): **tipo específico de SQL Server**, utilizado para identificadores únicos globales. Puede representarse como cadena o en formato binario.
- Delimitadores: Se utilizan para separar los diferentes componentes de una sentencia (palabras clave, identificadores, constantes, etc.).

Entre los delimitadores más comunes se encuentran:

- Paréntesis ()
- Comas (,)
- Espacios en blanco
- Punto y coma (;) para finalizar una sentencia

En resumen, las sentencias SQL combinan estos elementos -comandos, identificadores, constantes y delimitadores- siguiendo un orden lógico y sintáctico.

Una correcta comprensión de su estructura es esencial para formular consultas y operaciones precisas sobre las bases de datos.



2.4.2. Comandos SQL

Como se ha visto en el epígrafe anterior, las sentencias SQL están formadas, entre otros elementos, por palabras clave.

Algunas de estas palabras son comandos, que representan la acción principal que se desea realizar sobre la base de datos, y otras son cláusulas, que complementan o delimitan dicha acción.

Los comandos SQL son palabras clave que solicitan una operación específica dentro del sistema gestor, constituyendo la base de cualquier instrucción.

Estos comandos se agrupan en varias categorías o sublenguajes, según su función dentro del sistema.

Las categorías principales son:

DDL (Data Definition Language)

Comandos que se utilizan para definir, crear y modificar la estructura de la base de datos y sus objetos (tablas, vistas, índices, procedimientos, etc.).

Entre ellos tenemos:

- **CREATE:** crea objetos en la base de datos (tablas, vistas, esquemas, etc.).
- **DROP:** elimina objetos existentes.
- **ALTER:** modifica la estructura o propiedades de un objeto ya creado.
- **TRUNCATE:** elimina todos los registros de una tabla sin borrar su estructura.

Aunque afecta a los datos, se clasifica dentro del DDL porque actúa sobre la tabla como un todo y no permite operaciones selectivas fila a fila, a diferencia de DELETE. Su comportamiento interno, incluido el uso del registro de transacciones, depende del SGBD.

DML (Data Manipulation Language)

Comandos que se utilizan para consultar y manipular los datos almacenados en la base de datos.

Entre ellos tenemos:

- **SELECT:** consulta datos de una o varias tablas.
- **INSERT:** agrega nuevos registros.
- **UPDATE:** modifica datos existentes.
- **DELETE:** elimina registros específicos.
- **MERGE:** combina operaciones de inserción, actualización o eliminación en una sola instrucción, disponible en algunos SGBD como SQL Server y Oracle.



DCL (Data Control Language)

Comandos destinados a gestionar los permisos, privilegios y la seguridad de los objetos y usuarios de la base de datos.

Entre ellos tenemos:

- **GRANT:** otorga privilegios o permisos a un usuario o rol.
- **REVOKE:** revoca los privilegios previamente concedidos.

TCL (Transaction Control Language)

Comandos que permiten gestionar las transacciones dentro de la base de datos, garantizando la coherencia y recuperación de los datos.

Entre ellos tenemos:

- **COMMIT:** confirma una transacción y guarda permanentemente los cambios.
- **ROLLBACK:** revierte los cambios realizados durante la transacción actual.
- **SAVEPOINT:** establece un punto intermedio dentro de una transacción para poder deshacer los cambios solo hasta ese punto.

2.4.3. Cláusulas

Las cláusulas son los componentes que modifican o complementan a los comandos SQL, permitiendo definir condiciones, filtrar resultados, agrupar o establecer el orden de los registros obtenidos en una consulta.

Al igual que los comandos, son palabras clave reservadas que forman parte esencial de la estructura de una sentencia.

Aunque se estudiarán más adelante con mayor detalle, se presentan brevemente las principales:

FROM:

Permite especificar la tabla o tablas sobre las que se desea realizar la consulta.

Para poder mostrar campos en una sentencia SELECT, es obligatorio indicar la fuente de los datos mediante la cláusula FROM.

Es una cláusula obligatoria en las consultas de selección.



WHERE:

Define las condiciones que deben cumplir los registros para ser incluidos en el resultado.

Solo los registros que satisfacen la condición indicada aparecen en la salida de la consulta.

No es una cláusula obligatoria, pero se utiliza con frecuencia para filtrar datos.

GROUP BY:

Permite agrupar los registros seleccionados según el valor de uno o más campos.

Suele emplearse en combinación con funciones de agregado (como SUM, AVG, COUNT, MAX, MIN).

HAVING:

Se utiliza junto con **GROUP BY** para establecer condiciones sobre los grupos generados, especialmente cuando se aplican funciones de agregado.

A diferencia de WHERE, que filtra registros individuales antes del agrupamiento, HAVING filtra los resultados agrupados.

ORDER BY:

Permite ordenar los registros resultantes de una consulta en función de uno o varios campos, de forma ascendente (ASC) o descendente (DESC).

Es opcional, pero muy común en consultas de presentación o informes.



+ Info

El orden correcto en una sentencia es:

SELECT – FROM – WHERE – GROUP BY – HAVING – ORDER BY



2.4.3.1. Predicados

Un predicado es una expresión lógica o booleana que se evalúa con uno de tres posibles valores: TRUE (verdadero), FALSE (falso) o UNKNOWN (desconocido).

En SQL, un predicado representa la condición que se evalúa para cada fila de una tabla o conjunto de resultados, y determina si dicha fila cumple o no el criterio especificado.

Los predicados se utilizan principalmente en las cláusulas que requieren condiciones lógicas, como WHERE, HAVING, JOIN ON, CHECK o dentro de expresiones CASE.

Permiten filtrar registros, establecer restricciones o tomar decisiones condicionales dentro de una consulta.

Ejemplos de predicados:

- edad > 18
- nombre IS NOT NULL
- EXISTS (SELECT ...)
- salario BETWEEN 1000 AND 2000
- ciudad LIKE 'M%'

Principales operadores de predicado:

Los operadores que generan predicados son IN, NOT IN, EXISTS, NOT EXISTS, ANY, SOME, ALL, BETWEEN, LIKE, IS NULL y IS NOT NULL.

Su combinación con operandos produce condiciones evaluables dentro de una sentencia SQL.

Contextos de uso de los predicados:

- En la condición de búsqueda de las cláusulas WHERE y HAVING.
- En subconsultas, junto con operadores como EXISTS, IN, ANY o ALL.
- En restricciones (CHECK) y en condiciones de unión (JOIN ON).
- En expresiones condicionales, como CASE WHEN... THEN... END.



Operadores de predicado en subconsultas

- En subconsultas y uniones, se pueden utilizar ANY o SOME, ALL, IN o NOT IN, EXISTS o NOT EXISTS.

Modificadores de consulta

- Algunos sistemas gestores incorporan modificadores de consulta que no son predicados, como TOP, DISTINCT o ALL, los cuales afectan al número de filas devueltas o al tratamiento de duplicados, pero no constituyen condiciones lógicas evaluables como los predicados.
- En operaciones de combinación de consultas (por ejemplo, UNION ALL), el modificador ALL permite combinar resultados sin eliminar duplicados.



+ Info

El predicado TOP es propio de SQL Server, y se utiliza para limitar el número de filas devueltas por una consulta. En otros sistemas gestores se emplean alternativas como LIMIT (en MySQL, MariaDB, PostgreSQL y SQLite) o FETCH FIRST (en Oracle y DB2).

2.4.4. Operadores

Un operador es un símbolo o una combinación de símbolos que se utiliza para especificar una acción o comparación entre una o más expresiones dentro de una sentencia SQL.

Como ya hemos indicado anteriormente, se clasifican en diferentes tipos:

- Lógicos o booleanos.
- Aritméticos.
- De comparación.

2.4.4.1. Operadores lógicos o booleanos

Los operadores lógicos se utilizan para comprobar la veracidad o falsedad de una condición, por lo que el valor que devuelven es de tipo BOOLEAN, es decir: TRUE (verdadero), FALSE (falso) o UNKNOWN (desconocido).



Los operadores lógicos más comunes son: AND, OR y NOT.

Algunos sistemas gestores de bases de datos también implementan XOR (exclusive OR), aunque no forma parte del estándar SQL y su disponibilidad depende del SGBD (por ejemplo, está presente en MySQL, pero no en SQL Server o Oracle).

Las expresiones con operadores lógicos se evalúan de izquierda a derecha, respetando la prioridad de los operadores, que puede modificarse mediante paréntesis.

El resultado de una expresión lógica puede ser:

- TRUE (verdadero).
- FALSE (falso).
- UNKNOWN (desconocido).

Cuando interviene un valor NULL en la evaluación de una condición, el resultado suele ser UNKNOWN, ya que SQL utiliza lógica ternaria (verdadero, falso, desconocido) en lugar de lógica estrictamente binaria.

Por esta razón, los valores NULL deben gestionarse con precaución, utilizando funciones o condiciones explícitas como IS NULL o COALESCE(), para evitar resultados inesperados o comportamientos no deseados en consultas o aplicaciones.

Los valores nulos también pueden afectar la integridad de los datos si no se controlan correctamente, lo que contraviene los principios de normalización de bases de datos establecidos por Edgar F. Codd.

2.4.4.2. Operadores aritméticos

Los operadores aritméticos se utilizan para realizar operaciones matemáticas sobre valores numéricos.

Pueden aplicarse tanto a campos como a constantes y expresiones dentro de una sentencia SQL.

En la mayoría de los sistemas gestores, los operadores suma (+) y resta (–) también pueden emplearse con valores de tipo fecha y hora (datetime) para sumar o restar intervalos temporales.

OPERADOR	SIGNIFICADO
+	Suma. También puede utilizarse para sumar intervalos a valores de fecha/hora.
-	Resta. Permite restar valores numéricos o intervalos de tiempo.
*	Producto (multiplicación).
/	División. Devuelve el cociente resultante de la operación.



OPERADOR	SIGNIFICADO
%	Módulo: devuelve el resto de una división entera. Su comportamiento puede variar según el tipo de datos y el SGBD.
^	Exponenciación (potencia): Operador no estándar ANSI SQL. En algunos SGBD representa una operación bit a bit (XOR) y no la exponenciación. Para calcular potencias en SQL se utiliza habitualmente la función POWER(base, exponente).

2.4.4.3. Operadores de comparación

También denominados relacionales, los operadores de comparación se utilizan para evaluar la relación entre dos expresiones, determinando si son iguales, diferentes, mayores o menores entre sí.

El resultado de la comparación es siempre un valor booleano: TRUE, FALSE o UNKNOWN (cuando intervienen valores nulos).

Los operadores de comparación presentan limitaciones con los tipos de datos text, ntext e image, especialmente en SQL Server, donde estos tipos se encuentran en desuso y han sido sustituidos por VARCHAR(MAX), NVARCHAR(MAX) o VARBINARY(MAX).

OPERADOR	SIGNIFICADO
=	Igual
<>	Distinto (no igual) - operador estándar ANSI SQL
!=	Distinto (no igual) - alternativo en algunos SGBD como SQL Server o MySQL
>	Mayor
>=	Mayor o igual
!>	No mayor que - no forma parte del estándar, pero admitido por SQL Server
<=	Menor o igual
<	Menor
!<	No menor que - no estándar, reconocido por SQL Server



2.4.4.3.1. Operador like

En SQL, para buscar un patrón dentro de una columna de texto, se utiliza principalmente el operador LIKE, junto con los comodines % y _.

Estos comodines permiten realizar búsquedas flexibles de cadenas de caracteres dentro de una consulta.

La sintaxis básica es:

```
expresión [NOT] LIKE 'patrón';
```

Donde:

- Expresión: es una expresión SQL que se evalúa en una cláusula WHERE.
- Patrón: cadena de caracteres con la que se compara la expresión.

Para simplificar las búsquedas, el patrón puede incluir los siguientes caracteres comodín:

- % (porcentaje): reemplaza cualquier secuencia de caracteres, incluidos ninguno.
- _ (guion bajo): reemplaza un único carácter cualquiera.

El signo % se utiliza, por tanto, para sustituir cualquier número de caracteres, mientras que el _ reemplaza solo uno.

Sensibilidad a mayúsculas y minúsculas

El comportamiento del operador LIKE respecto a la distinción entre mayúsculas y minúsculas no depende del propio operador, sino de la collation definida en la base de datos o en la columna.

En algunos SGBD, como SQL Server, las comparaciones con LIKE no distinguen entre mayúsculas y minúsculas por defecto, mientras que en otros, como PostgreSQL, sí se distingue, salvo que se utilice el operador específico ILIKE.

Por tanto, el resultado de una comparación con LIKE puede variar entre sistemas gestores dependiendo de la configuración de la collation.



Vamos a poner varios ejemplos con la tabla siguiente llamada *usuarios*:

id	nombre	apellido
1	Victor	Campo
2	Vitorina	Arregui
3	Antonio	Pérez

```
SELECT nombre FROM usuarios WHERE nombre like "%to%";
```

Esta consulta devolverá los tres nombres, pues en los tres casos existe la coincidencia del patrón buscado, "to", al que le aplicamos el comodín %, al principio y al final del mismo, al encontrar en los tres nombres ese patrón nos devolverá los tres.

```
SELECT nombre FROM usuario WHERE nombre LIKE "Vi_to%";
```

Esta consulta devolverá solamente el nombre de **Victor** pues el patrón coincide solo coincide con este caso, el patrón conicidirá con Vi to y los comodines podrían ser cualquier caracter en tercera posición y cualesquiera caracteres en las últimas posiciones tras la "o".

Por último, pongamos otro ejemplo con otra tabla *productos*:

codigo	nombre	precio
A11	ratón	17€
A21	monitor	150€
A51	teclado	40€

Imaginemos que ejecutamos esta consulta en SQL Server:

```
SELECT nombre FROM productos WHERE codigo REGEXP '^A[1-3]1';
```



Nuestra consulta nos devolverá los resultados, *ratón* y *monitor*, pues el segundo caracter del código ha de estar entre los valores 1 y 3 incluidos ambos, A51, el código del teclado no lo está, así que no será listado.

Esta sintaxis con REGEXP no es válida en SQL Server, pero sí funciona en sistemas que admiten expresiones regulares, como MySQL, MariaDB, PostgreSQL o Oracle (mediante REGEXP_LIKE).

2.4.4.3.2. Operador IN

El operador IN permite especificar un conjunto de valores, y la consulta (o subconsulta) devolverá los registros cuyo valor coincida con uno de los elementos de ese conjunto.

Se aplica habitualmente sobre una sola columna. En algunos SGBD puede utilizarse sobre varias columnas mediante constructores de filas, aunque no es una característica universal.

El operador IN se emplea principalmente en la cláusula WHERE, aunque también puede utilizarse en otras como HAVING o en condiciones de unión (JOIN ON).

Los valores se separan por comas y se escriben entre comillas simples únicamente cuando son cadenas de caracteres.

La lista de valores puede indicarse de forma explícita o bien ser el resultado de una subconsulta mediante una sentencia SELECT.

Sintaxis:

```
SELECT campo/s FROM nombretabla WHERE (campo/s) IN ('valor1', 'valor2', 'valor3');
```

2.4.5. Funciones de agregado

Podemos efectuar operaciones sobre un conjunto de resultados y obtener un único valor agregado, como máximos, medias, etc., sobre un conjunto de valores.

Las funciones de agregado realizan un cálculo sobre un conjunto de valores que cumplen una determinada condición, devolviendo un solo valor calculado.

El resultado es una totalización global o por grupos cuando se utiliza la cláusula GROUP BY.



Hay diferentes funciones de agregación en SQL (cada SGBD puede añadir las suyas propias). Indicamos las 5 funciones básicas, que estudiaremos con mayor profundidad en la cláusula **GROUP BY**:

- **AVG**: devuelve el valor promedio de los valores de un campo concreto que especifiquemos, por tanto, solo se puede utilizar en columnas numéricas.

Con GROUP BY calcula el promedio de los valores de cada grupo.

- **COUNT**: Devuelve el número total de filas seleccionadas por la consulta.

Con GROUP BY cuenta el número de registros de cada grupo.

- **COUNT(*)** cuenta todas las filas, mientras que COUNT(columna) solo cuenta las filas en las que dicha columna no es NULL.

- **SUM**: Suma todos los valores del campo que especifiquemos. Sólo se puede utilizar en columnas numéricas.

Con GROUP BY devuelve la suma de los valores de cada grupo.

- **MAX**: Devuelve el valor máximo del campo especificado. Puede aplicarse a columnas numéricas, de texto o de fecha.

Con GROUP BY devuelve el valor máximo de cada grupo.

- **MIN**: Devuelve el valor mínimo del campo especificado. Puede aplicarse a columnas numéricas, de texto o de fecha.

Con GROUP BY devuelve el valor mínimo de cada grupo.

2.4.6. Sintaxis SQL y su precedencia

El orden lógico de ejecución de una sentencia SQL no coincide exactamente con el orden en que se escribe.

A efectos de interpretación por el sistema gestor, la precedencia de las operaciones es la siguiente:

1. Origen de los datos: se identifican las tablas o vistas que participan en la consulta (FROM / JOIN).
2. Filtro principal: se aplican las condiciones de filtrado sobre los datos (WHERE).
3. Agrupación: se agrupan los resultados según los campos indicados (GROUP BY).
4. Filtro sobre la agrupación: se filtran los grupos resultantes (HAVING).
5. Selección final: se determinan las columnas o expresiones a mostrar (SELECT).
6. Ordenación: se ordenan los resultados obtenidos (ORDER BY).



2.5. Transacciones

Una transacción es una unidad única de trabajo, de forma que:

- **Si una transacción tiene éxito**, todas las modificaciones de los datos realizadas durante la transacción se confirman y se convierten en una parte permanente de la B.D.
- **Si una transacción encuentra errores** y debe cancelarse o revertirse, se borran todas las modificaciones de los datos.

El ejemplo más claro y utilizado para comprender este concepto, es la operación de realizar una transferencia de dinero de una cuenta bancaria a otra, para lo cual se realizan 2 procesos:

- Restar la cantidad a transferir en la cuenta de origen.
- Sumar la cantidad transferida en la cuenta destino.

Deben realizarse los 2 procesos, si por algún error (corte en la comunicación, suministro eléctrico, etc.) la operación no se termina completamente (se realiza la resta, pero no la suma) se produciría inconsistencia en la información, y se perdería el rastro del dinero.

Por ello, las bases de datos relacionales cuentan con un control de transacciones, de forma que solo se graba si se realiza el proceso completo, y si no se ha completado se deja en el estado inicial.

El comportamiento de las transacciones se define mediante las propiedades ACID, que garantizan su fiabilidad:

- **Atomicity** (Atomicidad): la transacción se ejecuta completamente o no se ejecuta en absoluto.
- **Consistency** (Consistencia): el estado de la base de datos permanece válido antes y después de la transacción.
- **Isolation** (Aislamiento): las transacciones concurrentes no interfieren entre sí.
- **Durability** (Durabilidad): una vez confirmada, la transacción permanece registrada incluso ante fallos del sistema.

En SQL, el control de transacciones se realiza mediante las siguientes sentencias:

- **BEGIN TRANSACTION / START TRANSACTION**: inicia una transacción.
- **COMMIT**: confirma los cambios realizados y los hace permanentes.
- **ROLLBACK**: deshace todos los cambios desde el inicio de la transacción.

Estas sentencias están disponibles en los principales sistemas gestores de bases de datos (SQL Server, MySQL/MariaDB, PostgreSQL, Oracle y SQLite), aunque su comportamiento concreto puede presentar diferencias según el SGBD y su configuración, especialmente en aspectos como el modo autocommit o el nivel de aislamiento.



3. ANSI SQL. Subconjuntos de lenguajes

El lenguaje SQL permite, a través de un gran repertorio de sentencias, realizar diferentes funciones como:

- Consultar datos.
- Crear, actualizar, y eliminar tanto datos como elementos-objetos de la B.D.

De forma habitual, el lenguaje SQL se clasifica en diferentes grupos de lenguajes según su función:

- **Lenguaje de definición de datos (DDL).**

Incluye comandos para la definición, modificación y eliminación de estructuras de la base de datos, como tablas, vistas o esquemas.

- **Lenguaje de manipulación de datos (DML).**

Incluye comandos para consultar, insertar, modificar y eliminar los datos almacenados en las tablas.

- **Lenguaje de control de datos (DCL).**

Permite controlar el acceso a los datos contenidos en la Base de Datos.

- **Lenguaje de control de transacciones (TCL).**

Incluye comandos para la gestión de transacciones, como COMMIT, ROLLBACK y SAVEPOINT, que permiten confirmar o deshacer los cambios realizados en la base de datos.

La división en estos subconjuntos del lenguaje SQL permite gestionar distintos aspectos de las bases de datos, como:

- **Integridad y definición de vistas.**

Incluye comandos para definir restricciones de integridad y crear vistas. Las actualizaciones que violan dichas restricciones se rechazan.

- **Control de transacciones.**

SQL incluye comandos para confirmar o deshacer transacciones y gestionar puntos de guardado (por ejemplo, COMMIT, ROLLBACK y SAVEPOINT).

- **SQL incorporado y SQL dinámico.**

Hacen referencia a las distintas formas de integrar instrucciones SQL dentro de lenguajes de programación de propósito general, como C o Java.



- **Autorización.**

El DCL de SQL incluye comandos para especificar los derechos de acceso a las relaciones y a las vistas.

Algunos autores proponen otras clasificaciones no estandarizadas del lenguaje SQL, como:

- Lenguaje de control o procesamiento de transacciones (**TPL**).
- Lenguaje de control del cursor (**CCL**).

Son denominaciones utilizadas por algunos autores para agrupar determinados comandos, aunque no constituyen subconjuntos oficiales del estándar ANSI SQL.

4. DDL

Siglas del inglés **Data Definition Language**, (en español Lenguaje de definición de Datos).

DDL permite definir la estructura lógica de una base de datos (el esquema de la B.D.) y sus objetos

Hay que definir las características y restricciones de los atributos para optimizar el acceso a los datos de una tabla, definir las características y restricciones de los atributos, y muy importante, definir restricciones sobre el tipo de dato que puede contener la columna de una tabla.

Para realizar esta creación de la B.D, y definir su estructura lógica, se utilizan diferentes comandos, que realizan todo lo necesario como:

- **Modificación de la estructura de los objetos de la base de datos.**

Crear y definir nuevos objetos, así como poder modificarlos, renombrarlos y borrarlos. En objetos como:

- La propia **Base de Datos** (Crear, Borrar).
- **Tablas** (Crear, modificar su estructura, eliminar).
- **Campos:**
 - » Definir su dominio o modificarlo (ALTER).
 - » Definir campos autoincrementales (según el SGDB).
 - » Definir índices sobre uno o varios campos
 - » Definir restricciones (clave, obligatorio...).



- **Relaciones entre tablas de la Base de Datos.**

La definición de los campos como clave principal o ajena, relaciona las tablas entre sí.

- **Definición de vistas.**

Vamos a indicar un esquema de los comandos de DDL, clasificados por su función:

CREATE (crear)	Definición de la estructura de los objetos de la base de datos (bds). Permite crear bds, tablas, vistas, índices, etc.
ALTER (modificar)	Modifica la estructura de un objeto existente (por ejemplo, agregar o eliminar columnas o restricciones).
DROP (eliminar)	Elimina objetos de la bds (tablas, vistas, índices, etc.).
TRUNCATE (vaciar de contenido)	Elimina todos los registros de una tabla sin borrar su estructura. Más rápido que DELETE y sin posibilidad de usar ROLLBACK en algunos SGBD.
RENAME (renombrar)	Cambia el nombre de un objeto existente. Disponible en la mayoría de SGBD.
PRIMARY KEY, FOREIGN KEY, REFERENCES	DEFINEN las relaciones entre las tablas y garantizan la integridad referencial.
CHECK CONSTRAINT, NOT NULL	RESTRICCIONES sobre los dominios de las columnas de una tabla. Se usan para asegurar la integridad de los datos.

4.1. Modificación de la estructura de los objetos de la B.D.

Las bases de datos y sus objetos (tablas, vistas, índices, procedimientos, etc.) se crean, modifican o eliminan mediante los comandos del lenguaje DDL (Data Definition Language).

Estos comandos permiten definir la estructura lógica de la base de datos y mantener su diseño actualizado conforme evolucionan las necesidades del sistema.

4.1.1. Bases de Datos

Para crear o eliminar una Base de Datos, se utiliza **DATABASE**:

- **CREATE DATABASE:** Se utiliza para crear una base de datos.

Sintaxis: `CREATE DATABASE nombre_base_datos.`

Ejemplo: `CREATE DATABASE baseDeJugadores.`



- **DROP DATABASE:** Se utiliza para eliminar una base de datos.

Sintaxis: DROP DATABASE nombre_base_datos.

Ejemplo: DROP DATABASE baseDeJugadores.

Podemos comprobar si existe o no la B.D antes de borrarla, y si existe borrarla, utilizando la opción **IF EXISTS**.

Sintaxis: DROP DATABASE [IF EXISTS] nombre_basedatos.



+ Info

Para eliminar varias bases de datos con una sola declaración, se indica entre corchetes los nombres de las BB.DD. a eliminar, separados por comas.

Diferencias entre SGBD

- **MySQL / MariaDB:** DROP DATABASE [IF EXISTS] db1, db2; sí admite varios nombres separados por comas y la cláusula IF EXISTS. CREATE DATABASE permite CHARACTER SET y COLLATE.
- **PostgreSQL:** DROP DATABASE [IF EXISTS] nombre; admite IF EXISTS, no admite eliminar varias B.D. en una sola sentencia. CREATE DATABASE permite ENCODING, LC_COLLATE, LC_CTYPE, TEMPLATE.
- **SQL Server:** DROP DATABASE [IF EXISTS] nombre [, ...n] admite varios nombres separados por comas y la cláusula IF EXISTS (desde SQL Server 2016). En versiones anteriores se utilizaba una comprobación previa sobre sys.databases.
- **Oracle:** Operaciones de creación/eliminación de DATABASE son tareas de administración (no de aplicación). DROP DATABASE solo en modo MOUNT con privilegios adecuados; en entornos multitenant se usa DROP PLUGGABLE DATABASE ... INCLUDING DATAFILES.
- **SQLite:** No existe CREATE/DROP DATABASE; la "base de datos" es un archivo. Se usa ATTACH/DETACH DATABASE y la creación/eliminación se gestiona a nivel de fichero del sistema operativo.



4.1.2. Tablas

Para crear, borrar o eliminar la estructura de una Tabla de una Base de Datos, se utiliza **TABLE**:

- **CREATE TABLE**: Se utiliza para **crear** una tabla.

Pondremos el nombre de cada atributo seguido de su dominio.

Si el atributo es obligatorio (no puede tomar valores nulos), añadimos "NOT NULL" después del dominio.

Podemos indicar también, al crearla, la clave primaria (simple o compuesta) utilizando PRIMARY KEY, o hacerlo con posterioridad.

Sintaxis:

```
CREATE TABLE nombretabla (  
    atributo1 tipodato1 NOT NULL,  
    atributo2 tipodato2 NOT NULL,  
    atributo3 tipodato3 NOT NULL,  
    PRIMARY KEY (atributosClave)  
);
```

Ejemplo:

```
CREATE TABLE jugadores(  
    idJugador int NOT NULL,  
    nombre varchar(255),  
    apellido1 varchar(255),  
    apellido2 varchar(255),  
    edad int,  
    primary key(idJugador)  
);
```

**Resultado:**

jugadores				
idJugador	nombre	apellido1	apellido2	edad
*				

- **DROP TABLE:** Se utiliza para eliminar una tabla.

Sintaxis:

```
DROP TABLE nombreTabla;
```

Ejemplo: borramos la tabla jugadores.

```
DROP TABLE jugadores;
```

Podemos comprobar si existe o no la tabla antes de eliminarla y si existe eliminarla usando la opción IF EXISTS

```
DROP TABLE [IF EXISTS] nombreTabla;
```

- **TRUNCATE TABLE:**

Trunca (borra) todo el contenido de una tabla.

Internamente, el comando TRUNCATE borra la tabla y la vuelve a crear sin datos, por eso es mucho más rápido que el comando DELETE y no se permite el uso de la cláusula WHERE (borra todo).

Sintaxis:

```
TRUNCATE TABLE nombreTabla;
```



Diferencias entre SGBD

- **MySQL / MariaDB:** Admiten DROP TABLE [IF EXISTS] y TRUNCATE TABLE. Este último puede reiniciar los valores AUTO_INCREMENT.
- **PostgreSQL:** Soporta DROP TABLE [IF EXISTS] y TRUNCATE TABLE. Permite truncar varias tablas a la vez (TRUNCATE TABLE t1, t2;) y la opción CASCADE para tablas relacionadas.
- **SQL Server:** DROP TABLE IF EXISTS disponible desde SQL Server 2016. TRUNCATE TABLE no puede usarse si existen claves foráneas activas.
- **Oracle:** DROP TABLE nombre CASCADE CONSTRAINTS; elimina la tabla y sus dependencias. TRUNCATE TABLE requiere privilegios DROP y no puede revertirse con ROLLBACK.
- **SQLite:** DROP TABLE IF EXISTS es válido; no implementa TRUNCATE TABLE (se simula con DELETE FROM nombreTabla;).

4.1.3. Campos

Los campos se definen al crear la tabla, pero también se pueden añadir o eliminar posteriormente, así como cambiar su tipo de dominio (dependiendo del tipo), mediante la cláusula **ALTER TABLE**.

Además de tener un determinado tipo de dominio, se pueden establecer sobre ellos otras características y restricciones, como son:

- Definirlo como campo autoincremental.
- Crear índices sobre uno o varios campos (INDEX).
- Aplicarle restricciones (NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY o FOREIGN KEY, etc.).

Vamos a estudiar con más detalle estas operaciones sobre los atributos de una tabla.

Diferencias entre SGBD

- **MySQL / MariaDB:** permiten AUTO_INCREMENT, MODIFY COLUMN y CHANGE COLUMN. Las restricciones pueden definirse al crear o modificar la tabla.
- **PostgreSQL:** utiliza SERIAL o GENERATED AS IDENTITY para autoincrementos. Admite ALTER TABLE ... ALTER COLUMN TYPE para cambiar tipos de datos compatibles.
- **SQL Server:** usa IDENTITY para autoincrementales; no permite modificar el tipo de columna si afecta a datos incompatibles.
- **Oracle:** los autoincrementales se definen con secuencias y triggers (o GENERATED AS IDENTITY en versiones recientes).
- **SQLite:** solo admite un campo autoincremental por tabla y debe declararse como INTEGER PRIMARY KEY AUTOINCREMENT.



4.1.3.1. Alter table

Modifica la estructura de la tabla, puede añadir un atributo, cambiar su tipo de dominio, y borrarlo.

Por ejemplo, podemos añadir o eliminar campos, modificar el tipo de datos, o cambiar propiedades como claves o valores por defecto.

- **Añadir un atributo: ADD.**

Sintaxis:

```
ALTER TABLE nombreTabla ADD nombreAtributo dominio;
```

Ejemplo: añadimos el atributo posición tipo varchar

```
ALTER TABLE jugadores ADD posicion varchar(255);
```

Resultado:

jugadores					
idJugador	nombre	apellido1	apellido2	edad	posicion
1	Iker	Casillas	Fernández	37	
*					

- **Modificar el dominio de un atributo: MODIFY COLUMN /ALTER COLUMN.**

No siempre se puede modificar el dominio de una columna, dependerá de cómo está definido ese campo, en cuanto a tipo de dato y también si es un índice, clave, etc. Por ejemplo, reducir el tamaño de caracteres de una columna puede provocar que se trunquen los datos, perdiéndose información.

Dependiendo del SGBD que estemos usando el comando secundario puede ser MODIFY o ALTER.

No se puede modificar cualquier tipo de dominio, hay que tener en cuenta como están definidos.



La posibilidad de modificar el dominio de una columna depende del SGBD y de las restricciones asociadas. En muchos casos, antes de realizar el cambio puede ser necesario eliminar o modificar previamente determinados elementos, como:

- Columnas de tipos de datos especiales (por ejemplo, timestamp en algunos SGBD).
- Columnas que forman parte de una clave primaria o clave ajena (PRIMARY KEY / FOREIGN KEY).
- Columnas incluidas en índices, especialmente cuando se pretende reducir su longitud.
- Columnas sujetas a restricciones CHECK o UNIQUE.
- Columnas con valores predeterminados (DEFAULT) asociados.

Asimismo, en algunos SGBD, como SQL Server, existen conversiones específicas entre tipos grandes de datos, como text, ntext o image, hacia tipos más modernos como varchar(max), nvarchar(max) o varbinary(max).

Sintaxis:

```
ALTER TABLE nombreTabla ALTER COLUMN nombreAtributo nuevoDominio;
```

o

```
ALTER TABLE nombreTabla MODIFY COLUMN nombreAtributo nuevoDominio;
```

- **Eliminar un atributo: DROP.**

Sintaxis:

```
ALTER TABLE nombreTabla DROP COLUMN nombreAtributo;
```

Ejemplo: eliminar el atributo 'posicion'.

```
ALTER TABLE nombreTabla DROP posicion;
```

La palabra reservada COLUMN en DROP COLUMN es obligatoria en SQL estándar y en SQL Server; DROP nombreColumna solo es válido en algunos SGBD concretos (MySQL, PostgreSQL, Oracle)



Diferencias entre SGBD

- **MySQL / MariaDB:** utilizan ADD, MODIFY, CHANGE, y DROP COLUMN. Admiten ADD CONSTRAINT y DROP CONSTRAINT.
- **PostgreSQL:** usa ADD COLUMN, ALTER COLUMN TYPE, y DROP COLUMN. Permite renombrar columnas con RENAME COLUMN.
- **SQL Server:** usa ADD, ALTER COLUMN, y DROP COLUMN. No permite modificar tipos si hay datos incompatibles ni eliminar columnas referenciadas.
- **Oracle:** ADD, MODIFY, DROP COLUMN y RENAME COLUMN. Los cambios pueden requerir recompilar vistas o restricciones dependientes.
- **SQLite:** ALTER TABLE tiene soporte limitado: permite solo ADD COLUMN; no puede eliminar ni modificar tipos (hasta versiones recientes que soportan DROP COLUMN experimentalmente).

4.1.3.2. Campos autoincrementales

La instrucción **CREATE SEQUENCE** permite ir asignando un valor correlativo a un atributo para cada nuevo registro que se crea. Se suelen utilizar para el campo definidos como clave primaria de la tabla.

Se puede especificar:

- Valor inicial. (si no se especifica será 1)
- Incremento.

Sintaxis:

```
CREATE SEQUENCE nombreAutoIncremental START WITH valorInicial INCREMENT BY  
incremento;
```

Ejemplo:

```
CREATE SEQUENCE idJugador START WITH 1 INCREMENT=1;
```



+ Info

Hoy en día, la mayoría de los SGBD incluyen mecanismos específicos para definir campos **autoincrementales**, sin necesidad de crear una secuencia de forma manual.

Diferencias entre gestores SGBD:

Ponemos a continuación distintos modos de manejar el autoincremento en distintos sistemas gestores:

SQL estándar (ANSI/ISO):

```
nombre_columna tipo_dato GENERATED ALWAYS AS IDENTITY (START WITH 1 INCREMENT BY 1);
```

SQL Server:

Usa el modificador IDENTITY (inicio, incremento).

```
idJugador INT IDENTITY (1,1) PRIMARY KEY;
```

MySQL / MariaDB:

AUTO_INCREMENT directamente en la definición de la columna.

```
idJugador INT AUTO_INCREMENT PRIMARY KEY;
```

PostgreSQL:

Tradicionalmente usa los tipos especiales SERIAL o BIGSERIAL, que internamente crean una secuencia.

Desde la versión 10, recomienda GENERATED AS IDENTITY conforme al estándar SQL.



```
idJugador SERIAL PRIMARY KEY;
```

Oracle:

Antes requería definir manualmente una secuencia y un trigger para asignar el valor.

Desde la versión 12c, admite directamente GENERATED AS IDENTITY, coincidiendo con PostgreSQL a partir de su décima versión:

```
idJugador NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
```

SQLite:

Solo permite un campo autoincremental por tabla, definido como:

```
idJugador INTEGER PRIMARY KEY AUTOINCREMENT;
```

4.1.3.3. Definir campos como índices

Definir campos de una tabla como índices, sirve para facilitar la obtención de los datos cuando se realiza una consulta. (Al igual que el índice de un libro, permite localizar rápidamente un capítulo por el número de página).

Se emplea un índice SQL para poder recuperar datos de una base de datos de una manera más rápida.

En la gran mayoría de sistemas de gestión de base de datos, se crea al definir los índices una estructura de datos que se almacena en la misma base de datos que las tablas indexadas. Dicha estructura contiene punteros a los contenidos de una tabla organizada en un orden específico, para ayudar a la base de datos a optimizar las consultas. Mediante la organización de los valores de una columna en una estructura ordenada, se permite el acceso rápido a los registros de la tabla mediante búsquedas eficientes en dicha estructura.

El objetivo principal de un índice es acelerar la recuperación de información y mejorar el rendimiento de las consultas.



Sin embargo, presenta una desventaja: consume espacio adicional en disco y puede ralentizar las operaciones de escritura (INSERT, UPDATE, DELETE), ya que el índice debe actualizarse cada vez que cambia un valor indexado.

Al realizar una búsqueda, sin un índice se debe recorrer toda la tabla secuencialmente para encontrar un registro, la existencia de un índice permite el acceso directo haciendo las búsquedas más rápidas y eficientes.

Por ello deben definirse como índices únicamente aquellos campos por los que se realizan frecuentemente operaciones de búsqueda, filtrados o uniones (WHERE, JOIN, ORDER BY, etc.).

El índice organiza los datos, por lo general, en una estructura arbórea multinivel (B-Tree) que funciona como un "índice de índices", permitiendo al servidor descartar rápidamente grandes secciones de datos irrelevantes y localizar exactamente los registros buscados con mínimos accesos a disco, aunque algunos sistemas emplean otras estructuras como Hash o Bitmap según el tipo de dato o la operación.

Vamos a indicar algunas características de los índices:

- Pueden estar formados por más de 1 campo (multicolumna).
- Una tabla se indexa por un campo (o varios): puede tener uno o más índices.
- Un índice es una estructura de datos que optimiza el acceso a la información en una tabla mediante la organización de sus entradas según los valores de una o más columnas.
- Los índices son una estructura que acelera las consultas de lectura ordenando sus propias filas por un campo, optimizando el acceso a la tabla de datos.
- Tipos principales: Clustered (define orden físico) y Non-clustered (estructura separada).
- Son transparentes para los usuarios.
- Los índices se usan para distintas operaciones, como son:
 - Buscar registros rápidamente.
 - Recuperar registros de otras tablas empleando **"JOIN"**.
 - Cuando existe un índice en las columnas utilizadas en la cláusula **WHERE** de una consulta, el uso de la cláusula JOIN o la cláusula **ORDER BY**, resulta más rápido y eficiente.



+ Info

Indexar una tabla optimiza el acceso a los datos y mejora el rendimiento, especialmente en tablas con miles de registros.



Creamos un índice mediante la instrucción **CREATE INDEX**:

- El comando **CREATE INDEX** designa uno o varios campos como índice.
- Por defecto, los índices no son únicos, aunque pueden declararse como tales con la opción **UNIQUE**.
- Tampoco es necesario que las columnas sean **NOT NULL**.
- Puede haber varios en la misma tabla, y le podemos dar un nombre o dejar que el sistema le asigne uno por defecto.
- Podemos especificar si queremos que aplique un orden ascendente (**ASC**) o descendente (**DESC**).

Sintaxis:

```
CREATE INDEX nombreIndice ON tabla (atributo [orden]);
```

Ejemplo:

```
CREATE INDEX idxIdJugador ON jugadores (idJugador ASC);
```

También podemos crear índices con condiciones adicionales, denominados índices parciales en PostgreSQL e índices filtrados en SQL Server. Estos permiten incluir únicamente las filas que cumplen una condición, optimizando el rendimiento y reduciendo el tamaño del índice.

```
CREATE INDEX idxNombre ON jugadores (nombre) WHERE nombre IS NOT NULL;
```



Atención

Los campos definidos como clave primaria crean automáticamente un índice único (normalmente clustered) en esa tabla. Los campos definidos como clave ajena (foreign key) NO crean automáticamente ningún índice; es una buena práctica crearlo manualmente para optimizar las operaciones JOIN. La clave ajena referencia a la clave primaria (que sí es un índice) de otra tabla en la base de datos.



Diferencias por SGBD

- **MySQL / MariaDB:** usan por defecto B-Tree para la mayoría de índices; FULLTEXT para búsquedas de texto y HASH para tablas MEMORY. El índice primario (PRIMARY KEY) siempre es clustered en InnoDB.
- **PostgreSQL:** soporta B-Tree, Hash, GIN (para arrays o texto), GiST, y SP-GiST. Permite índices parciales y multicolumna con expresiones.
- **SQL Server:** distingue clustered y non-clustered, y permite índices filtrados, columnstore y full-text. Una tabla puede tener solo un índice clustered.
- **Oracle:** utiliza B-Tree por defecto; admite índices bitmap (para columnas con pocos valores distintos) y function-based (sobre expresiones).
- **SQLite:** admite índices únicos o no únicos, pero no soporta índices parciales hasta versiones recientes (3.8+), donde los permite con WHERE.

4.1.3.4. Restricciones a los campos de las tablas

Se pueden especificar unas condiciones (restricciones) que deben cumplir los campos de una tabla, limitando los valores que puede recibir una columna de una tabla.

Las restricciones se pueden definir cuando creamos la tabla (CREATE TABLE) o posteriormente con la sentencia ALTER TABLE.

Las restricciones se crean o eliminan mediante la cláusula **CONSTRAINT**.

La cláusula **CONSTRAINT** se utiliza con los comandos **CREATE TABLE** y **ALTER TABLE**

Los tipos comunes de restricciones son:

- **PRIMARY KEY** (Claves primarias).

Designa el campo como clave principal de la tabla, identificando de forma única cada registro en una tabla.

Recordemos que una tabla solo puede tener una clave primaria (PRIMARY KEY), y los valores deben ser únicos (UNIQUE) y no nulos (NOT NULL).

Una clave primaria puede estar formada por más de un campo de la tabla (clave compuesta).

Puede ser parte de un dato real o un campo artificial sin relación directa con el contenido.

Puede definirse al crear la tabla (CREATE TABLE) o posteriormente con ALTER TABLE.



- **FOREIGN KEY** (Claves Ajenas).

Designa el campo como clave externa de la tabla, indicando una relación con otra tabla de la base de datos.

Un campo definido como FOREIGN KEY, solo puede contener valores que ya existan en la clave primaria (PRIMARY KEY) de la tabla referenciada, asegurando la integridad referencial. Una FOREIGN KEY no crea automáticamente un índice; se recomienda crearlo manualmente para optimizar las operaciones JOIN.

- **NOT NULL** (Obligatoriedad).

Se utiliza para restringir (evitar) que una columna pueda tener valores nulos (NULL), ya que de forma predeterminada sí que puede ser NULL.

- **UNIQUE** (Unicidad).

Asegura que todos los valores en una columna sean distintos.

El campo debe tener valores únicos, y se acepta que tenga valores nulos. Si se intenta agregar un registro con un valor ya existente, aparecerá un mensaje de error. El comportamiento respecto a múltiples valores NULL depende del SGBD.

Puede haber varios campos UNIQUE en la misma tabla.

- **CHECK** (Verificación de condiciones).

Se utiliza para asegurar que todos los valores en una columna cumplan ciertas condiciones (por ejemplo, mayor que 10 en un campo de tipo dato entero).

- **DEFAULT** (Valores por defecto).

Proporciona un valor predeterminado a una columna. Si al insertar un registro no se especifica un valor, se asignará el valor indicado en la restricción DEFAULT.

Las restricciones se pueden definir a nivel de columna o de tabla.

Restricción en creación de tabla:

```
CONSTRAINT nombre_restriccion
{
    PRIMARY KEY (columna1 [, columna2, ...])
    | UNIQUE (columna1 [, columna2, ...])
    | FOREIGN KEY (columna_local1 [, columna_local2, ...])
      REFERENCES tabla_externa (columna_ref1 [, columna_ref2, ...])
    | CHECK ( condición )
}
```



Ejemplos prácticos:

```
CREATE TABLE orden (  
    id_orden INT NOT NULL,  
    id_cliente INT NOT NULL,  
    fecha DATE,  
    CONSTRAINT pk_orden PRIMARY KEY (id_orden),  
    CONSTRAINT fk_cliente FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente)  
);
```

- Para crear una restricción **en varios campos** (restricción compuesta).

```
CREATE TABLE detalle_venta (  
    id_venta INT,  
    id_producto INT,  
    cantidad INT,  
    CONSTRAINT pk_detalle PRIMARY KEY (id_venta, id_producto),  
    CONSTRAINT fk_detalle_venta FOREIGN KEY (id_venta)  
        REFERENCES venta(id_venta),  
    CONSTRAINT fk_detalle_producto FOREIGN KEY (id_producto)  
        REFERENCES producto(id_producto)  
);
```

Diferencias entre SGBD

Las restricciones están implementadas de forma general en todos los sistemas gestores de bases de datos, aunque existen diferencias relevantes en su comportamiento según el motor utilizado.

- En **MySQL y MariaDB**, las restricciones siguen el estándar SQL, pero presentan particularidades. Las claves foráneas (FOREIGN KEY) solo se aplican si la tabla utiliza el motor InnoDB y no crean índices de forma automática, por lo que conviene definirlos manualmente. Las restricciones CHECK fueron ignoradas en versiones anteriores a la 8.0, donde sí comenzaron a aplicarse correctamente. El resto de restricciones (PRIMARY KEY, UNIQUE, NOT NULL, DEFAULT) funcionan conforme al estándar, y DEFAULT admite funciones como CURRENT_TIMESTAMP.
- En **SQL Server**, el soporte de restricciones es completo. Las claves primarias (PRIMARY KEY) crean automáticamente un índice clustered si no existe otro definido. Las claves foráneas no generan índices automáticos, aunque su uso es altamente recomendado para optimizar las uniones. Las restricciones CHECK y DEFAULT son totalmente funcionales y se evalúan en el momento de inserción o actualización de datos.



- **PostgreSQL** ofrece una implementación muy fiel al estándar SQL y una de las más flexibles. Permite definir todas las restricciones, incluyendo CHECK, con expresiones complejas e incluso funciones. Además, admite claves primarias compuestas, índices parciales y validación inmediata o diferida de las claves foráneas, lo que aporta un control avanzado sobre la integridad referencial.
- En **Oracle**, todas las restricciones del estándar están soportadas y con gran madurez. Las claves primarias y únicas crean índices de forma automática, las foráneas mantienen integridad referencial estricta, y las restricciones CHECK permiten condiciones sofisticadas. Las restricciones pueden definirse y activarse o desactivarse temporalmente, lo que proporciona flexibilidad en tareas de mantenimiento o migración.
- **SQLite** presenta un comportamiento más limitado. Aunque admite todas las restricciones sintácticamente, algunas -como FOREIGN KEY o CHECK- solo se aplican si la base de datos se ha compilado o configurado con la opción `foreign_keys=ON`. Además, solo puede existir una clave primaria por tabla, y las restricciones no siempre se nombran ni se validan de forma estricta.

4.2. Relaciones entre las tablas: FOREIGN KEY y REFERENCES

Las tablas de una base de datos se pueden relacionar entre sí mediante sus claves.

Para ello, en una tabla (tabla hija) se define la clave externa (FOREIGN KEY) que creará una relación con la PRIMARY KEY de otra tabla (tabla padre). La cláusula REFERENCES es la que indica este enlace específico.

- **FOREIGN KEY.**

Indica qué campo actúa como clave foránea en la tabla hija. Este campo contendrá valores que deben existir previamente en la tabla padre, garantizando así la integridad referencial entre ambas tablas.

- **REFERENCES.**

Especifica la tabla y el campo de la tabla padre a los que apunta la clave foránea (FOREIGN KEY). Ambas cláusulas se definen conjuntamente dentro de la creación o modificación de la tabla.

Sintaxis:

```
FOREIGN KEY(campo_foraneo_tabla_hija) REFERENCES tabla_padre(campo_pk_tabla_padre)
```

Donde:

- `campo_foraneo_tabla_hija`: es el campo de la tabla hija que actuará como FOREIGN KEY.
- `tabla_padre`: es la tabla que contiene la primary key a la que apunta la FOREIGN KEY.
- `campo_pk_tabla_padre`: es el campo PRIMARY KEY de la tabla padre al que se enlaza la FOREIGN KEY.



Ejemplo:

Supongamos la tabla Alumnos, con el campo id_alumno definido como PRIMARY KEY.

La tabla Matriculas incluye el campo alumno_id, que actuará como FOREIGN KEY, estableciendo la relación entre ambas tablas:

```
FOREIGN KEY (alumno_id) REFERENCES Alumnos(id_alumno)
```

De este modo, la base de datos asegura que no se pueda registrar una matrícula con un alumno_id que no exista en la tabla Alumnos, manteniendo la coherencia e integridad de los datos.

Acciones referenciales: ON DELETE y ON UPDATE

Cuando se define una clave foránea (FOREIGN KEY), es posible indicar qué debe ocurrir en la tabla hija cuando el registro relacionado en la tabla padre se modifica o se elimina.

Estas reglas de comportamiento se denominan acciones referenciales y se especifican mediante las cláusulas ON DELETE y ON UPDATE.

Sintaxis:

```
FOREIGN KEY (campo_foraneo)
REFERENCES tabla_padre (campo_pk)
ON DELETE acción
ON UPDATE acción;
```

Opciones posibles

- **CASCADE:** si se elimina o modifica un registro en la tabla padre, los cambios se propagan automáticamente a las filas relacionadas en la tabla hija (por ejemplo eliminar un alumno elimina también todas sus matrículas).
- **SET NULL:** si el registro de la tabla padre se elimina o modifica, el valor de la clave foránea en la tabla hija se reemplaza por NULL (si el campo lo permite).
- **SET DEFAULT:** establece el valor predeterminado definido en la columna foránea cuando el registro de la tabla padre cambia o se elimina.



- **RESTRICT / NO ACTION:** impide la eliminación o actualización del registro padre si existen filas dependientes en la tabla hija. En la práctica, NO ACTION (según el estándar) y RESTRICT (según implementación) tienen un comportamiento equivalente en la mayoría de SGBD.

```
FOREIGN KEY (alumno_id)
REFERENCES Alumnos(id_alumno)
ON DELETE CASCADE
ON UPDATE CASCADE;
```

4.3. Definición de vistas CREATE VIEW

Las vistas, se almacenan como metadatos en la propia base de datos pero no almacenan datos, no almacenan ninguna tabla sino que devuelven el resultado de la consulta cuando son invocadas.

Una vista no almacena datos físicamente, solo los muestra de la forma que deseemos, y cada vez que se realiza la consulta a una vista, el sistema de la Base de Datos actualiza esa vista (la crea cada vez) de forma que se muestran siempre los datos reales existentes en ese momento.

Muestran siempre datos reales de una o varias tablas.

Cuando tenemos creada una vista, cada vez que queramos mostrarla utilizaremos la instrucción: **SELECT * FROM [nombreLista]**

Las vistas deben tener nombres únicos y puede incluir datos de una o varias tablas.

CREATE VIEW (Crear una vista)

La sintaxis para la creación de una vista de una tabla es la siguiente:

```
CREATE VIEW [nombreLista] AS
SELECT atributo1, atributo2... atributoN
FROM nombreTabla
WHERE condición;
```



Ejemplo en nuestra Tabla Jugadores.

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	

Creamos una vista llamada "apodosDefensas" que muestre los datos de edad y apodo de aquellos jugadores que son defensas.

```
CREATE VIEW [apodosDefensas] AS
SELECT edad, apodo FROM jugadores WHERE posición='defensa';
```

Indicamos que se muestre la vista.

```
SELECT * FROM [apodosDefensas];
```

Resultado:

edad	apodo
46	Correcaminos
32	Tarzán de Camas
40	Tiburón



Modificar una vista. REPLACE VIEW

Podemos modificar una vista ya existente, por ejemplo, quitando o agregando los campos a mostrar, o cambiando la condición.

Para ello utilizamos la misma sintaxis que para crear la vista, pero sustituyendo **CREATE VIEW** por **CREATE OR REPLACE VIEW**, según el SGBD. Esta instrucción actualiza la definición de la vista si ya existe, sin necesidad de eliminarla previamente.

Eliminar una vista. DROP VIEW

Para eliminar una lista utilizaremos:

```
DROP VIEW [nombreLista];
```

Diferencias entre SGBD

Aunque la sintaxis básica de CREATE VIEW, CREATE OR REPLACE VIEW y DROP VIEW es común en la mayoría de los sistemas, existen diferencias relevantes en su comportamiento y en las características admitidas:

- **MySQL / MariaDB**

Permiten crear y reemplazar vistas con CREATE OR REPLACE VIEW.

Las vistas no almacenan datos físicamente, pero pueden volverse "no actualizables" si incluyen agregaciones, funciones o uniones (JOIN).

Admiten la cláusula opcional WITH CHECK OPTION, que impide insertar o modificar datos que no cumplan las condiciones definidas en la vista.

Ejemplo:

```
CREATE OR REPLACE VIEW activos AS SELECT * FROM empleados WHERE activo = 1 WITH CHECK OPTION;
```

- **PostgreSQL**

Admite también CREATE OR REPLACE VIEW, con funcionamiento similar. Además, permite crear vistas materializadas mediante CREATE MATERIALIZED VIEW, que almacenan los resultados físicamente para consultas más rápidas. Estas deben actualizarse manualmente con REFRESH MATERIALIZED VIEW nombreVista;.

Las vistas normales no son actualizables si incluyen subconsultas complejas, agregaciones o funciones.



- **SQL Server**

Soporta la creación con `CREATE VIEW`, pero no con `CREATE OR REPLACE VIEW`.

Para modificar una vista debe emplearse `ALTER VIEW`.

Puede incluir la opción `WITH SCHEMABINDING`, que vincula la vista a la estructura de las tablas subyacentes, impidiendo cambios en ellas mientras la vista exista.

Ejemplo:

```
CREATE VIEW vistaActivos WITH SCHEMABINDING AS SELECT nombre FROM dbo.empleados
WHERE activo = 1;
```

Oracle Database

Permite `CREATE OR REPLACE VIEW` con semántica idéntica a la de PostgreSQL o MySQL.

Además, soporta vistas materializadas (`CREATE MATERIALIZED VIEW`) que pueden.

Ejemplo:

La opción `WITH CHECK OPTION` también está disponible. actualizarse de forma manual o automática.

```
CREATE OR REPLACE VIEW empleados_activos AS SELECT * FROM empleados WHERE
estado='ACTIVO' WITH CHECK OPTION;
```

SQLite

Implementa `CREATE VIEW` y `DROP VIEW`, pero no `CREATE OR REPLACE VIEW`.

No soporta vistas materializadas ni actualizables.

Todas las vistas son virtuales y de solo lectura.

5. DML

Siglas del inglés **D**ata **M**anipulation **L**anguage, (en español LDD, siglas de Lenguaje de Manipulación de Datos).

Las instrucciones DML permiten consultar o modificar el contenido de los datos almacenados en la B.D.



En general, las operaciones básicas de manipulación de datos que podemos realizar con SQL se denominan operaciones CRUD (**C**read, **R**ead, **U**pdate, **D**elete).

Operación	Acción	Sentencia SQL	Modifica datos	Subconjunto SQL
CREATE	Crear objetos (tablas, vistas...)	CREATE	Sí	DDL
READ	Leer datos	SELECT	No	DQL (Data Query Language)
UPDATE	Actualizar registros	UPDATE	Sí	DML
DELETE	Eliminar registros	DELETE	Sí	DML
INSERT	Insertar registros	INSERT	Sí	DML
MERGE	Insertar o actualizar condicionalmente	MERGE	Sí	DML

La sentencia MERGE (también conocida como upsert) permite agregar o modificar filas de forma condicional, seleccionando registros de una tabla origen para insertarlos o actualizarlos en una tabla destino o vista.

Está disponible en sistemas como SQL Server, Oracle y PostgreSQL (desde la versión 15). En MySQL, puede emularse mediante INSERT ... ON DUPLICATE KEY UPDATE.

En función de si modifican los datos o no, las consultas se clasifican en dos tipos:

- Consultas de acción: Modifican los datos almacenados. Comandos: INSERT, UPDATE, DELETE, MERGE.
- Consultas de selección: No modifican los datos, solo los muestran o filtran. Comando: SELECT (DQL, Data Query Language).

Diferencias entre los SGDB

- **SQL Server:** admite todas las sentencias DML estándar, incluido MERGE, plenamente compatible con el estándar ANSI SQL:2008. También permite usar cláusulas como OUTPUT para devolver los registros afectados tras una operación DML.
- **Oracle:** soporta MERGE desde hace varias versiones con amplia flexibilidad (permite condiciones complejas y subconsultas). Incluye además la sentencia INSERT ALL para realizar múltiples inserciones en una sola operación.



- **PostgreSQL:** implementa MERGE a partir de la versión 15. Ofrece soporte completo para RETURNING, que permite devolver valores de filas insertadas o actualizadas sin ejecutar una consulta adicional.
- **MySQL / MariaDB:** no incluye MERGE como tal, pero permite el mismo comportamiento con INSERT ... ON DUPLICATE KEY UPDATE o REPLACE INTO. RETURNING está disponible desde MySQL 8.0.19 y MariaDB 10.5.
- **SQLite:** soporta las operaciones básicas (INSERT, UPDATE, DELETE), pero no MERGE. Dispone de la extensión INSERT OR REPLACE para lograr un efecto similar.

5.1. AS (alias)

Se utiliza en las consultas para que, en lugar de mostrar el nombre del atributo definido en la tabla, se muestre un nombre alternativo (alias) indicado en la propia consulta.

Para ello se indica primero el nombre del atributo y a continuación la palabra clave AS, seguida del nombre que deseamos que se aparezca. Este nuevo nombre solo existe el tiempo de ejecución de la consulta.

Ejemplo:

Teniendo la tabla jugadores (idJugador, nombre, apellido1, apellido2, edad, posicion, apodo).

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	



Instrucción:

Queremos mostrar el campo *apodo* con el nombre *alias*, junto con *nombre* y *apellido1*.

SELECT indica que queremos seleccionar campos, y FROM indica de qué tabla queremos seleccionar dichos campos). Se estudiará más adelante.

```
SELECT apodo AS alias, nombre, apellido1 FROM jugadores;
```

Resultado: nos muestra solo los campos que hemos indicado (apodo, nombre y apellido1), sustituyendo el nombre del campo "apodo" por "alias".

alias	nombre	apellido1
San Iker	Iker	Casillas
Pulpo	Luis Miguel	Arconada
Correcaminos	Sergi	Barjuan
Tarzán de Camas	Sergio	Ramos
Tiburón	Carles	Puyol
Zizou	Zinédine	Yazid
Gusiluz	Andrés	Iniesta
Santillana	Carlos	Alonso
	Julio	Salinas



+ Info

Podemos definir el orden en que se muestran los atributos.

En el ejemplo anterior, al colocar apodo en primer lugar, este aparece el primero en el resultado.

El orden de las columnas mostradas siempre sigue el orden definido en la sentencia SELECT.



En SQL, la palabra clave AS es opcional. se puede escribir:

```
SELECT apodo alias FROM jugadores;
```

Sin embargo, usarla mejora la claridad y legibilidad, por lo que se recomienda mantenerla, especialmente en consultas complejas o en entornos corporativos (SQL Server, PostgreSQL, Oracle, MySQL, SQLite... todos la admiten).

Diferencias entre SGBD

El uso de AS es opcional en la mayoría de los sistemas (SQL Server, MySQL, MariaDB, PostgreSQL y Oracle).

En SQLite, también es opcional, aunque recomendado por claridad.

Algunos sistemas, como Oracle, PostgreSQL o MySQL, permiten definir alias tanto para columnas como para tablas; sin embargo, en SQL Server no puede usarse AS para alias de tablas, solo para columnas.

5.2. Consultas de selección

El objetivo de las **Sentencias de consulta** de datos (DQL, Data Query Language), es visualizar, seleccionar y organizar los datos de las tablas que componen la base de datos.

Para ello se generan consultas que filtren datos y/o los ordenen de la forma concreta en que se especifica.

El comando principal es SELECT, indica que queremos ejecutar una sentencia de SQL de selección. En algunos contextos, SELECT también puede actuar como cláusula dentro de una instrucción mayor (por ejemplo, en una subconsulta o vista).

SELECT muestra los resultados de la consulta que realiza en forma de tabla, y esta tabla resultante puede ser incorporada en una aplicación o bien utilizarse de forma interactiva.

Para conocer la versión de la Base de Datos, podemos ejecutar la instrucción SELECT:

- En ORACLE:

```
SELECT * FROM v$VERSION;
```



- En PostgreSQL o MySQL:

```
SELECT version();
```



Atención

Para conocer la versión de la Base de Datos, podemos ejecutar la instrucción SELECT, con la cláusula FROM indicando V\$VERSION, quedando la instrucción a ejecutar así:

```
SELECT * FROM V$VERSION
```

Hay diferentes tipos de consultas, pudiéndolas clasificarlas del siguiente modo:

- **Básicas** (SELECT, filtrado y ordenación).
- **Subconsultas** (incluye correlacionadas).
- **Agrupamiento** (GROUP BY y HAVING).
- **Combinaciones** (JOINS) entre tablas.
- **Operaciones de conjuntos**: UNION / UNION ALL / INTERSECT / EXCEPT (MINUS en Oracle).
- **CTE** (WITH) para consultas recursivas o estructuradas.
- **Funciones ventana** (OVER/PARTITION BY/ORDER BY) para cálculos por filas.

5.2.1. Consultas básicas

Lo que deseamos al realizar estas consultas, es obtener el contenido de unos determinados campos de una tabla, ordenar los resultados, y también indicar que los campos que nos muestre cumplan unos requisitos indicados.

Seleccionar uno o varios campos de una o varias tablas.

Para ello se utiliza el comando SELECT, que es la sentencia básica del lenguaje de consulta, junto con la cláusula FROM.



Podemos indicar que estos resultados se muestren de diferentes formas:

- Que no aparezcan resultados repetidos: con la cláusula **DISTINCT**.
- Ordenar los registros de forma ascendente o descendente, mediante la cláusula **ORDER BY**.
- Mostrar el resultado de un campo realizando previamente un cálculo sobre su valor (cálculos sobre atributos: **SUM**, **AVG**, **COUNT**, etc.).
- Indicar los requisitos que deben cumplir los datos que queremos obtener mediante la cláusula **WHERE**.

5.2.1.1. Seleccionar campos (**SELECT** con cláusula **FROM**)

SELECT indica los campos a consultar y la cláusula **FROM** indica de qué tabla queremos realizar la consulta. (Donde están los datos que queremos).

Sintaxis:

```
SELECT <nombreCampos> FROM <nombreTabla>;
```

Donde:

- **SELECT**: es la sentencia.
- **nombreCampos**: indicamos el nombre o nombres de campos que queremos obtener, se separan con una coma.

Si queremos obtener todos los campos de la tabla, en lugar de escribirlos todos, podemos utilizar el símbolo asterisco (*).

- **FROM**: es la cláusula para indicar de qué tabla queremos consultar los datos.
- **nombreTabla**: es el nombre de la tabla en la que se encuentran los datos que deseamos obtener.

Ejemplo: Teniendo la tabla jugadores.



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	

Instrucción:

```
SELECT nombre, edad FROM jugadores;
```

Resultado:

nombre	edad
Iker	37
Luis Miguel	64
Sergi	46
Sergio	32
Carles	40
Zinédine	46
Andrés	34
Carlos	66
Julio	55



Diferencias entre SGDB

En algunos sistemas gestores de bases de datos, como Oracle, los nombres de tablas y campos no distinguen entre mayúsculas y minúsculas si se crean sin comillas dobles. En cambio, en PostgreSQL, los identificadores se convierten por defecto a minúsculas, lo que puede afectar si se usan comillas al definirlos. Conviene mantener una convención clara (por ejemplo, siempre en minúsculas) para evitar errores.

5.2.1.1.1. DISTINCT

El predicado DISTINCT se utiliza para eliminar los valores duplicados en los resultados de una consulta.

Se indica antes de los nombres de columna, para evitar que se seleccionen filas duplicadas.

Teniendo nuestra tabla jugadores:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	

- **Ejemplo** donde queremos que se muestren todas las posiciones que hay en nuestra tabla jugadores.

```
SELECT posicion FROM jugadores;
```



- **Resultado:** nos muestra todos los valores del atributo posicion en la tabla. Como podemos observar, cada posición aparece tantas veces como filas tiene la tabla.

posicion
portero
portero
defensa
defensa
defensa
centrocampista
centrocampista
delantero
delantero

- Ejemplo donde queremos que se muestre el listado de los distintos (sin duplicados) valores de posición, utilizaremos la siguiente consulta:

```
SELECT DISTINCT posicion FROM jugadores;
```

- **Resultado:** Ahora ya no se repiten los valores, por tanto, no tenemos el mismo número de filas que tiene la tabla (Si no hubiera valores repetidos, entonces sí coincidiría el resultado con el número de filas de la tabla).

posicion
centrocampista
defensa
delantero
portero



Diferencias entre SGDB

En PostgreSQL, además de DISTINCT, existe la sintaxis DISTINCT ON (columna) que permite seleccionar la primera fila de cada grupo según un criterio de ordenación.

En todos los SGDB, los valores NULL también se consideran duplicados entre sí al aplicar DISTINCT. Por tanto, si existen varias filas con NULL en la misma columna, solo se mostrará una de ellas.

5.2.1.1.2. ORDER BY

Para ordenar los resultados obtenidos de la consulta de una tabla se utiliza la cláusula ORDER BY.

Admite dos parámetros:

- **ASC** (orden ascendente) es el valor predeterminado.
- **DESC** (orden descendente).

Ejemplo donde queremos que se muestren los jugadores ordenados por edad de mayor a menor.

```
SELECT * FROM jugadores ORDER BY edad DESC;
```

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
8	Carlos	Alonso	González	66	delantero	Santillana
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
9	Julio	Salinas	Fernández	55	delantero	
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
1	Iker	Casillas	Fernández	37	portero	San Iker
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas



Recuerda

Si no hubiéramos indicado DESC, (SELECT * FROM jugadores ORDER BY edad) el resultado sería ordenado de menor a mayor, ya que el valor por defecto es ASC.

Es posible ordenar por más de una columna, por ejemplo:

ORDER BY posicion ASC, edad DESC;

Ordena primero por posición alfabéticamente y, dentro de cada posición, por edad de mayor a menor.

Diferencias entre SGDB

- En **MySQL** y **PostgreSQL**, los valores NULL aparecen al principio en orden ascendente y al final en descendente.
- En **Oracle** y **SQL Server**, el comportamiento puede variar, pero puede modificarse explícitamente con las opciones NULLS FIRST o NULLS LAST.

5.2.1.1.3. Cálculos sobre atributos

Podemos efectuar cálculos sobre los atributos que queremos mostrar.

Por ejemplo, si queremos obtener la edad que tenían hace 10 años, y mostrar el resultado llamando a ese campo resultante edad2008 (es decir, un alias) utilizaríamos la siguiente sentencia:

```
SELECT nombre, apellido1, apellido2, edad, edad - 10 AS edad2008 FROM jugadores;
```

Resultado:

nombre	apellido1	apellido2	edad	edad2008
Iker	Casillas	Fernández	37	27
Luis Miguel	Arconada	Echarri	64	54



nombre	apellido1	apellido2	edad	edad2008
Sergi	Barjuan	Esclusa	46	36
Sergio	Ramos	García	32	22
Carles	Puyol	Saforcada	40	30
Zinédine	Yazid	Zidane	46	36
Andrés	Iniesta	Luján	34	24
Carlos	Alonso	González	66	56
Julio	Salinas	Fernández	55	45

En la mayoría de los SGBD (MySQL, PostgreSQL, SQL Server), el uso de AS es opcional; se puede escribir simplemente `edad - 10` `edad2008`. Sin embargo, se recomienda incluirlo siempre para mejorar la legibilidad.



ORACLE

En Oracle, cuando un alias contiene espacios o caracteres especiales, debe escribirse entre comillas dobles.

```
SELECT edad - 10 AS "Edad hace 10 años" FROM
jugadores;
```

5.2.1.2. Indicar requisitos (Cláusula WHERE)

La cláusula WHERE indica los requisitos que deben cumplir los datos que queremos obtener.

WHERE se utiliza, para establecer la condición (o condiciones) que han de cumplir los registros de la tabla que serán seleccionados. Actúa como un filtro: determina qué registros cumplen la condición y se mostrarán, y cuáles no.



Pueden utilizarse diferentes tipos de operadores, los operadores lógicos básicos, y otros que añaden más utilidades de gran ayuda:

- **operadores de comparación:**

- > (Mayor).
- >= (Mayor o igual).
- < (Menor).
- <= (Menor o igual).
- = (Igual).
- <> o != (Distinto).

Las condiciones son expresiones lógicas que pueden devolver TRUE, FALSE o UNKNOWN (cuando interviene NULL).

- **IS [NOT] NULL:**

Permite filtrar registros en función de si un campo tiene o no valor.

- **IS NULL:** mostrará las filas que **NO** tengan valores en un determinado campo.
- **IS NOT NULL:** mostrará las filas que **SÍ** tengan valores en un determinado campo.



Recuerda

NULL representa la ausencia de valor, no un cero, ni un espacio en blanco, ni una cadena vacía.

- **LIKE:**

Junto con la cláusula WHERE, permite buscar valores que coincidan con un patrón determinado dentro de una columna.

Se utilizan caracteres comodín para definir el patrón:

- "_": respresenta un único carácter cualquiera.
- [rango]: coincide con cualquier carácter del rango indicado (por ejemplo [a-c] o [abc]).



- [^rango]: selecciona los caracteres que no están dentro del rango.
- %: representa una cadena de cualquier longitud, incluso vacía.
- #: coincide con cualquier dígito. (en algunos SGBD como Access).

- **BETWEEN:**

Se utiliza para especificar un intervalo de valores.

- **Operador IN, ALL Y ANY:**

(Ya estudiados anteriormente)

- **Operadores lógicos:**

Podemos combinar varias condiciones simples con funciones lógicas como OR, AND y NOT.

Diferencias entre SGBD

- En **MySQL** el comportamiento de LIKE depende de la collation: por defecto suele ser NO sensible a mayúsculas y minúsculas.
- En **PostgreSQL**, el operador LIKE es sensible a mayúsculas y minúsculas; para búsquedas no sensibles se utiliza ILIKE.
- En **Oracle y SQL Server**, el comportamiento depende del tipo de colación configurado en la base de datos.

Recuerda que cualquier comparación con NULL (por ejemplo, campo = NULL) no devuelve resultados, ya que NULL no se evalúa como valor. Debe usarse siempre IS NULL o IS NOT NULL.

5.2.1.2.1. Sintaxis y ejemplos

Sintaxis:

```
SELECT nombreCampos FROM nombreTabla WHERE condicion;
```

Ejemplos partiendo de nuestra tabla base jugadores:



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	

Ejemplo WHERE con operador <

Queremos que se muestren los jugadores cuya edad sea menor de 40.

```
SELECT * FROM jugadores WHERE edad<40;
```

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz

Ejemplo WHERE con IS NULL

Queremos que se muestren solo las filas en las que el atributo apodo es NULL (no tiene valor).

```
SELECT * FROM jugadores WHERE apodo IS NULL;
```



Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
9	Julio	Salinas	Fernández	55	delantero	

Ejemplos WHERE con LIKE

- **Ejemplo 1:** queremos que se muestren los jugadores cuyo nombre contenga la cadena "Sergi".

```
SELECT * FROM jugadores WHERE nombre LIKE 'Sergi';
```

Resultado: muestra como resultado solo la fila cuyo nombre coincide totalmente con la cadena especificada.

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos

- **Ejemplo 2:** queremos que se muestren los jugadores cuyo nombre contenga la cadena "Sergi+un solo carácter".

```
SELECT * FROM jugadores WHERE nombre LIKE 'Sergi_';
```

Resultado: muestra como resultado solo la fila cuyo nombre coincide con la cadena especificada Sergi y cualquier otro carácter (sólo uno).

Si tuviéramos un jugador con nombre, por ejemplo Sergis o Sergiw también aparecería.

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas



- **Ejemplo 3:** queremos que se muestren los jugadores cuyo apellido1 contenga: "un carácter cualquiera que sea", "a", "3 caracteres cualesquiera que sean", "a", "s".

```
SELECT * FROM jugadores WHERE apellido1 LIKE '_a _ _as';
```

Resultado: muestra el jugador con apellido Salinas, que coincide con lo especificado.

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
9	Julio	Salinas	Fernández	55	delantero	

- **Ejemplo 4:** queremos que se muestren los jugadores cuyo apellido1 contenga: "un carácter cualquiera que sea", "a", "cualquier longitud de caracteres cualesquiera que sean", "a", "s".

```
SELECT * FROM jugadores WHERE apellido1 LIKE '_a%s';
```

Resultado: muestra el jugador con apellido "Casillas" y "Salinas", que coincide con lo especificado.

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
9	Julio	Salinas	Fernández	55	delantero	

En el **Ejemplo 3** solamente se muestran los apellidos que tienen tres caracteres entre "a" y "a". En este caso, como Casillas tiene cuatro no aparece.

En el **Ejemplo 4**, como usamos "%", servirá cualquier número de caracteres, por lo que aparecen los dos.

Ejemplo WHERE con BETWEEN (para especificar un intervalo de valores)

Queremos que se muestren los jugadores cuya edad esté comprendida entre 30 y 40 (ambos inclusive).

```
SELECT * FROM jugadores WHERE edad BETWEEN 30 AND 40;
```



Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz

Ejemplo WHERE con IN

Con IN se especifica que el atributo indicado debe contener un valor dentro del conjunto indicado por IN.

Ejemplo donde queremos que se muestren todas las filas cuya posición sea "defensa" o "centrocampista", que son los valores que estamos indicando dentro del IN.

```
SELECT * FROM jugadores WHERE posicion IN("defensa","centrocampista");
```

Resultado: muestra todas las filas cuya posición es "defensa" o "centrocampista".

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz

Ejemplo con operadores lógicos

Combinamos varias condiciones simples con funciones lógicas como OR, AND y NOT.

Ejemplo donde queremos que se muestren todas las filas cuyos jugadores que sean defensas o centrocampistas, además estén entre los 30 y 40 años (ambos inclusive).



```
SELECT * FROM jugadores WHERE posicion IN("defensa","centrocampista") AND edad BETWEEN 30 AND 40;
```

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz

5.2.2. Subconsultas

Una consulta puede contener otra consulta anidada a la que llamamos subconsulta.

En ocasiones, una consulta SELECT no es suficiente para expresar todas las condiciones que necesitamos, por lo que necesitamos utilizar las subconsultas.

A la hora de crear una consulta anidada, es necesario tener en cuenta dos conceptos muy importantes:

- La subconsulta siempre se ejecuta antes que la consulta principal y su resultado es usado por la consulta principal.
- Las subconsultas deben ponerse entre paréntesis.

En función del resultado que devuelve una subconsulta, podemos distinguir 2 tipos:

- **Monorregistro:** devuelve un solo registro.

Utilizan operadores de comparación que sólo devuelven un resultado.

- **Multirregistro:** devuelve más de un registro.

Utilizan cláusulas que comparan grupos de registro:

- **IN.** Devuelve verdadero si se encuentra en la lista obtenida de la subconsulta.
- **ALL.** Devuelve verdadero si la condición se cumple con todos los registros de la lista devuelta por la subconsulta.
- **ANY.** Devuelve verdadero si la condición se cumple con algún registro de la lista devuelta por la subconsulta.

El operador **NOT** podrá utilizarse en todos los casos para obtener el efecto contrario.



5.2.2.1. Ejemplos de Subconsultas

De nuevo, todos los ejemplos son sobre nuestra tabla "jugadores".

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	

Ejemplo 1: consultas anidadas

Queremos mostrar todos los jugadores que tienen la misma posición que el jugador de mayor edad. Para ello necesitaremos realizar una consulta con subconsultas (consultas anidadas).

La consulta a realizar sería:

```
SELECT * FROM jugadores WHERE posicion = (SELECT posicion FROM jugadores WHERE
edad = (SELECT MAX(edad) AS EdadMax FROM jugadores));
```

Lo vemos, paso a paso:

- Primero vamos a ver cómo es la consulta para saber cuál es la edad más alta (que usaremos cómo subconsulta).

```
SELECT MAX(edad) AS EdadMax FROM jugadores;
```

Busca en la columna edad cuál es la edad mayor (MAX) y la muestra con el nombre EdadMax que hemos indicado como alias (AS).



Resultado:

EdadMax
66

- A continuación, vamos a ver cómo es la consulta para saber la posición de ese jugador, por tanto, entre paréntesis está la consulta anterior como subconsulta:

(también lo usaremos como subconsulta)

```
SELECT posicion FROM jugadores WHERE edad = (SELECT MAX(edad) AS EdadMax FROM jugadores);
```

Busca la posición de los jugadores que cumplen la condición de tener 66 años.

Resultado:

posición
delantero

- Ahora que ya sabemos las consultas anteriores, realizamos la consulta completa, que nos dará el resultado que queríamos (mostrar todos los jugadores que tienen la misma posición que el jugador de mayor edad).

Consultamos todos los jugadores que tengan esta posición (de nuevo, la consulta anterior va entre paréntesis como subconsulta).

```
SELECT * FROM jugadores WHERE posicion = (SELECT posicion FROM jugadores WHERE edad = (SELECT MAX(edad) AS EdadMax FROM jugadores));
```

Resultado final:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	



Ejemplo 2: IN

Queremos averiguar las edades máximas de cada posición y queremos ver qué jugadores coinciden con alguna de esas edades (aunque sea de distinta posición).

La consulta a realizar sería:

```
SELECT * FROM jugadores WHERE edad IN (SELECT MAX(edad) AS EdadMax FROM jugadores GROUP BY posicion);
```

Lo vemos, paso a paso:

- Primero averiguamos las edades máximas, utilizando un alias "EdadMax) para mostrarlas:

```
SELECT MAX(edad) AS EdadMax FROM jugadores GROUP BY posicion;
```

- Veamos con detenimiento el funcionamiento de **MAX**.

Hacemos grupos por posición:

- Para posición **portero**, las edades son **37** y **64**.
De este grupo, la mayor es **64**.
- Para posición **defensa**, las edades son **46**, **32** y **40**.
De este grupo, la mayor es **46**.
- Para posición **centrocampista**, las edades son **46**, y **34**.
De este grupo, la mayor es **46**.
- Para posición **delantero**, las edades son **66** y **55**.
De este grupo, la mayor es **66**.

Resultado:

EdadMax
66
46



EdadMax
46
66

Nota: Si el ejercicio consistiera en obtener únicamente estos datos, en la consulta podríamos DISTINCT para evitar duplicados, por tanto, sólo se mostraría una línea con el número 46: SELECT DISTINCT posicion FROM jugadores). También podrías añadir ORDER BY para que lo mostrara ordenado ascendente o descendente.

- A continuación, buscamos todos los jugadores que tengan 64, 46 o 66 años.

Recuerda que IN devuelve verdadero si se encuentra en la lista obtenida de la subconsulta.

```
SELECT * FROM jugadores WHERE edad IN (SELECT MAX(edad) AS EdadMax FROM jugadores GROUP BY posicion);
```

Resultado final:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
8	Carlos	Alonso	González	66	delantero	Santillana

Ejemplo 3: ALL

Queremos mostrar los jugadores que tienen una edad inferior, a la que tienen los jugadores de mayor edad en cada posición.

La consulta a realizar sería:

```
SELECT * FROM jugadores WHERE edad < ALL (SELECT MAX(edad) AS EdadMax FROM jugadores GROUP BY posicion);
```



Lo vemos, paso a paso:

- Como en el ejemplo anterior, primero averiguamos las edades máximas:

```
SELECT MAX(edad) AS EdadMax FROM jugadores GROUP BY posicion;
```

Resultado:

EdadMax
64
46
46
66

- A continuación, buscamos todos los jugadores cuya edad sea menor que 64, 46 o 66.

Recuerda que ALL devuelve verdadero si la condición se cumple con todos los registros de la lista devuelta por la subconsulta.

```
SELECT * FROM jugadores WHERE edad < ALL (SELECT MAX(edad) AS EdadMax FROM jugadores GROUP BY posicion);
```

Resultado final:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz



Ejemplo 4: ANY

Queremos consultar todos los jugadores que estén en una posición diferente a aquellas que tengan dos jugadores.

La consulta a realizar sería:

```
SELECT * FROM jugadores WHERE posicion NOT IN (SELECT posicion FROM jugadores
GROUP BY posicion HAVING COUNT(*) = 2);
```

Lo vemos, paso a paso:

- En primer lugar, obtenemos las posiciones que solo tienen dos jugadores:

```
SELECT posicion FROM jugadores GROUP BY posicion HAVING COUNT(*) =2;
```

HAVING indica la condición que se debe cumplir, en este caso, que la posición debe tener 2 jugadores (COUNT).

Veamos con detenimiento el funcionamiento de COUNT (Cuenta el número de registros de cada grupo):

Hacemos grupos por posición:

- Para el grupo de posición **portero**, hay 2 jugadores.
- Para el grupo de posición **defensa**, hay 3 jugadores.
- Para el grupo de posición **centrocampista**, hay 2 jugadores.
- Para el grupo de posición **delantero**, hay 2 jugadores.

Se muestran por tanto las posiciones que tienen 2 jugadores.

Resultado:

posición
portero
centrocampista
delantero



- A continuación, mostramos todos los jugadores que no están en alguna de esas posiciones.

Recuerda que ANY devuelve verdadero si la condición se cumple con algún registro de la lista devuelta por la subconsulta.

```
SELECT * FROM jugadores WHERE posicion <> ANY (SELECT posicion FROM jugadores
GROUP BY posicion HAVING COUNT(*) =2);
```

Fíjate que estamos indicando <> **ANY**, (distinto de verdadero), por el resultado es el de los jugadores que no están en la subconsulta.

Resultado final:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón

5.2.2.2. Agrupamiento de registros: Cláusulas GROUP BY y HAVING

En ocasiones, es necesario obtener datos agrupados según los valores de una o varias columnas especificadas.

A modo introductorio, vamos a ver, que, ante múltiples filas de una tabla, podemos utilizar las cláusulas:

- **GROUP BY** (se utiliza con la sentencia SELECT).

Para indicar cómo agrupar.

Tras GROUP BY:

- **HAVING**: filtra los grupos resultantes de GROUP BY (actúa como un "WHERE" aplicado a grupos).
- **ORDER BY**: ordena las filas (o los grupos ya calculados) en la salida final por el o los campos se le indique. Se puede usar en cualquier SELECT que devuelva varias filas, haya o no agregados.

Tras ORDER BY:

- **ASC** o **DESC** son modificadores de ordenamiento que ordenarán ascendente o descendientemente (dependiendo del modificador elegido), las filas en base al campo de ordenamiento.



Veamos de nuevo las 5 funciones básicas de agregación, añadiendo alguna información:

- **COUNT.**

Devuelve el número total de filas seleccionadas por la consulta.

Con GROUP BY devuelve el número de elementos encontrados en un conjunto.

Diferenciamos **2 sintaxis diferentes:**

- **COUNT ([ALL | DISTINCT] expresión)**

En esta sintaxis:

- » **ALL** indica a la función COUNT () que se aplique a todos los valores.

ALL es el valor predeterminado.

- » **DISTINCT** indica a la función COUNT() que devuelva el número de valores únicos no nulos.

- » **Expresión** puede ser casi cualquier columna, salvo tipos de datos grandes como TEXT, NTEXT o IMAGE (en SQL Server, ya en desuso).

No se puede utilizar una subconsulta o una función agregada en la expresión.

- » **COUNT (*)**

Con esta sintaxis, devuelve el número total de filas del conjunto resultado, incluyendo duplicados y valores NULL.

Cuenta todas las filas del conjunto de resultados, incluso aquellas en las que haya valores NULL. No debe confundirse con COUNT(columna), que ignora los valores NULL de la columna especificada.

- **AVG.**

Devuelve el valor promedio de los valores de un campo concreto que especifiquemos, por lo que sólo se puede utilizar en columnas numéricas.

Con GROUP BY calcula el promedio de los valores de cada grupo.

- **SUM:**

Suma todos los valores del campo que especifiquemos. Sólo se puede utilizar en columnas numéricas.

Con GROUP BY devuelve la suma de los valores de cada grupo.

SUM y AVG solo se aplican a columnas numéricas.



- **MAX:**

Devuelve el valor máximo (más alto) del campo que especifiquemos, sirve de ayuda para acotar campos por cifras concretas, al igual que MIN).

Con GROUP BY devuelve el valor máximo de cada grupo.

- **MIN:**

Devuelve el valor mínimo del campo que especifiquemos.

Con GROUP BY devuelve el valor mínimo de cada grupo.

MAX y MIN, además de en números, también pueden usarse en fechas y en cadenas de texto, donde devuelven el valor mayor o menor según el orden cronológico o lexicográfico.

Por ejemplo, si tenemos una empresa de cuidado de caballos, y en una tabla los datos de cada caballo (id, nombre, edad, color, etc.) y una clave foránea con el id del cliente (id_cliente), dueño del animal, y queremos saber cuántos caballos estamos cuidando de cada cliente, realizaríamos una consulta agrupada por id_cliente, aplicando una función de agregación (por ejemplo, COUNT) para contar cuántos registros pertenecen a cada cliente.

```
SELECT id_cliente, COUNT(*) AS numero_caballos FROM caballos GROUP BY id_cliente;
```

En este ejemplo estaríamos utilizando la cláusula GROUP BY, y la función de agregado COUNT(*) que contaría las filas devueltas en cada grupo.



+ Info

El estándar SQL exige que todas las columnas que aparecen en la lista de selección (SELECT) y que no forman parte de una función de agregación, deben incluirse en la cláusula GROUP BY

5.2.2.2.1. Cláusula GROUP BY

La cláusula GROUP BY permite agrupar filas según el valor de una o varias columnas.

La cláusula **GROUP BY** agrupa las filas de una tabla en función del campo que le indiquemos, y muestra el dato que le hayamos solicitado en la consulta sobre ese conjunto que ha agrupado.



GROUP BY agrupa los registros y calcula los valores agregados por grupo; el resultado puede mostrarse en cualquier orden si no se especifica ORDER BY. Recuerda que ORDER BY, puede ordenar de forma ascendente (ASC) que es el valor por defecto, o de forma descendente (DESC).

Se puede también utilizar con más de un campo (nombre de columna), en cuyo caso hay que separar los nombres de cada columna por una coma.

Entonces podemos crear una consulta SQL, que nos realice agrupaciones, por ejemplo, en nuestra tabla de jugadores, si agrupamos por el atributo posición tendríamos cuatro grupos: portero, defensa, centrocampista y delantero. Y posteriormente podríamos hacer operaciones sobre los registros de cada uno de los cuatro grupos.

Veamos algunos ejemplos sobre nuestra tabla jugadores:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	

Ejemplo 1: Queremos saber la media de edad (AVG) mostrando el dato con el nombre de campo edadMedia (AS) de los jugadores que tienen la misma posición (GROUP BY).

El resultado puede mostrarse en un orden distinto según el SGBD, ya que GROUP BY no garantiza ningún orden.

```
SELECT posicion, AVG(edad) AS edadMedia FROM jugadores GROUP BY posicion;
```



Resultado (el orden de las filas puede variar si no se utiliza ORDER BY):

posicion	edadMedia
portero	50,5
defensa	39,33333333333333
centrocampista	40
delantero	60,5



+ Info

Pasos realizados internamente

Agrupar posiciones con la suma de sus edades y calcular la media:

Portero: $37 + 64 = 101$ $AVG (101 / 2) = 50,50$

Defensa: $46 + 32 + 40 = 118$ $AVG (118 / 3) = 39,33$

Centrocampista: $46 + 34 = 80$ $AVG (80 / 2) = 40$

Delantero: $66 + 55 = 121$ $AVG (121 / 2) = 60,50$

Si la columna por la que se agrupa contiene NULL, GROUP BY crea un grupo para ese valor. La diferencia viene en COUNT: con COUNT(*) se cuentan todas las filas del grupo (incluidos los NULL); con COUNT(columna) solo se cuentan las filas donde esa columna NO es NULL, de modo que el grupo de NULL da 0. El grupo siempre lo crea GROUP BY; COUNT únicamente determina cuántas filas de ese grupo se contabilizan.

La función de agregación COUNT admite como argumento el asterisco () o una columna/expresión. COUNT() cuenta todas las filas del conjunto resultado, haya o no valores NULL. COUNT(columna) cuenta solo las filas donde columna no es NULL. Las cadenas vacías () sí cuentan como no nulas en la mayoría de SGBD; en Oracle, " se trata como NULL y no se cuenta.

Si no hubiésemos usado GROUP BY, nos habría dado la media de entre todos los jugadores.

Podríamos haber indicado ORDER BY para que el resultado se mostrara ordenado por edadMedia.



Ejemplo 2: Ahora queremos saber cuántos jugadores hay en cada tipo de posición mostrando el dato con el nombre de NumeroJugadores (AS). Agrupamos por posición.

```
SELECT posicion, COUNT(posicion) AS NumeroJugadores FROM jugadores GROUP BY posicion;
```

Recuerda que dentro de COUNT no estamos indicando ALL por ser el valor por defecto.

posicion	NumeroJugadores
centrocampista	2
defensa	3
delantero	2
portero	2



+ Info

Dentro del COUNT podríamos haber puesto un * y el resultado sería el mismo en este caso, ya que la columna posicion no contiene valores NULL. Si existieran valores NULL, COUNT(posicion) no los contaría, mientras que COUNT(*) sí contaría todas las filas del grupo.

5.2.2.2.2. Cláusula HAVING

Se utiliza para especificar una condición que deben cumplir los grupos de resultados obtenidos tras una agregación.



+ Info

WHERE se aplica a las filas y HAVING a grupos.

La cláusula HAVING se agregó a SQL porque WHERE no puede aplicarse después de la agregación, y por tanto no puede filtrar resultados agregados.

Ejemplo: al igual que en el ejemplo anterior, queremos saber cuántos jugadores hay en cada tipo de posición mostrando el dato con el nombre de NumeroJugadores (AS), pero esta vez solo queremos que se muestren las posiciones que tengan más de 2 jugadores. (Utilizamos COUNT(*))

```
SELECT posicion, COUNT(*) AS NumeroJugadores FROM jugadores GROUP BY posicion
HAVING COUNT(*)>2;
```

posicion	NumeroJugadores
defensa	3

En la **cláusula HAVING** normalmente se comparan funciones de agregado, y se utiliza habitualmente junto con GROUP BY. Puede emplearse sin GROUP BY solo cuando la consulta devuelve un único grupo implícito, es decir, cuando se usan funciones de agregación sin columnas no agregadas.

5.2.3. Consultas de UNION

La sentencia **UNION** es utilizada para combinar los resultados de dos o más sentencias SELECT, que deben tener cómo resultado el mismo número de columnas, con tipos de datos compatibles y en el mismo orden.

Existen dos opciones de uso de la instrucción UNION, con o sin el modificador ALL:

- **UNION.**

Se seleccionan valores distintos, no aparecerán las filas repetidas.



- **UNION ALL.**

Al usar el modificador ALL, se seleccionan todos los valores, mostrando las filas repetidas, y la consulta es más rápida (no tiene que comparar valores duplicados).

La sintaxis es, para unión de dos o más tablas:

```
consulta1 UNION consulta2 UNION consultaN
```

Donde:

- ConsultaX puede ser cualquiera de las siguientes:
 - Instrucciones SELECT.
 - Una consulta SELECT (incluidas vistas).
 - Una tabla, utilizando la sintaxis TABLE nombreTabla (según el estándar SQL y algunos SGBD).

Lógicamente puesto que une consultas, puede utilizarse la cláusula GROUP BY y/o HAVING, y con ORDER BY solo al final de toda la unión, nunca en las consultas individuales.



+ Info

También se puede utilizar para combinar tablas independientes, indicando el nombre de dicha tabla precedido de la palabra TABLE.

O combinar tablas con consultas SELECT.

Ejemplos: Teniendo 2 tablas con la misma estructura llamadas jugadores y leyendas, y queremos realizar una consulta que nos muestre el contenido de ambas.

Tabla jugadores:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	

Tabla leyendas:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
20	Edson	Arantes	do Nascimento	77	delantero	Pelé
21	Diego	Armando	Maradona	57	delantero	Pelusa
22	Franz Anton	Beckenbauer		72	defensa	El Káiser

Ejemplo 1: queremos mostrar el contenido de las 2 tablas.

```
SELECT * FROM jugadores UNION SELECT * FROM Leyendas;
```

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
1	Iker	Casillas	Fernández	37	portero	San Iker
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas
5	Carles	Puyol	Saforcada	40	defensa	Tiburón



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
8	Carlos	Alonso	González	66	delantero	Santillana
9	Julio	Salinas	Fernández	55	delantero	
20	Edson	Arantes	do Nascimento	77	delantero	Pelé
21	Diego	Armando	Maradona	57	delantero	Pelusa
22	Franz Anton	Beckenbauer		72	defensa	El Káiser

Ejemplo 2: igual que en el ejemplo 1, queremos mostrar el contenido de las 2 tablas, pero mostrando los resultados ordenados por edad.

Para ordenarlas por edad, hay que anidarlas en otra sentencia SELECT para indicar que lo ordene. La instrucción del ejemplo 1 se ha convertido en subconsulta.

```
SELECT * FROM (SELECT * FROM jugadores UNION SELECT * FROM Leyendas) ORDER BY edad;
```

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
4	Sergio	Ramos	Garcia	32	defensa	Tarzán de Camas
7	Andrés	Iniesta	Luján	34	centrocampista	Gusiluz
1	Iker	Casillas	Fernández	37	portero	San Iker
5	Carles	Puyol	Saforcada	40	defensa	Tiburón
6	Zinédine	Yazid	Zidane	46	centrocampista	Zizou
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos
9	Julio	Salinas	Fernández	55	delantero	
21	Diego	Armando	Maradona	57	delantero	Pelusa



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo
8	Carlos	Alonso	González	66	delantero	Santillana
22	Franz Anton	Beckenbauer		72	defensa	El Káiser
20	Edson	Arantes	do Nascimento	77	delantero	Pelé

Ejemplo 3: queremos mostrar el contenido de las 2 tablas, donde la posición sea delantero.

```
SELECT * FROM jugadores WHERE posicion = 'delantero'
UNION
SELECT * FROM Leyendas WHERE posicion = 'delantero';
```

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo
9	Julio	Salinas	Fernández	55	delantero	
21	Diego	Armando	Maradona	57	delantero	Pelusa
8	Carlos	Alonso	González	66	delantero	Santillana
20	Edson	Arantes	do Nascimento	77	delantero	Pelé

Diferencias entre SGBD

En todos los SGBD (SQL Server, Oracle, PostgreSQL, MySQL/MariaDB y SQLite) se exige que las consultas combinadas devuelvan el mismo número de columnas, en el mismo orden y con tipos de datos compatibles.

Sin embargo, en Oracle y PostgreSQL, cuando se usan operaciones de conjuntos (UNION, INTERSECT, etc.), el ORDER BY que ordena el resultado combinado se coloca al final de toda la unión. Un ORDER BY dentro de cada SELECT solo tiene sentido/efecto si ese SELECT se encapsula como subconsulta y limita filas (por ejemplo, Oracle: FETCH FIRST; PostgreSQL: LIMIT/OFFSET).

En SQL Server y MySQL el ORDER BY debe colocarse tras la última unión si se desea ordenar el conjunto completo de resultados.



En SQLite, la ordenación incluida dentro de una subconsulta unida puede ser ignorada si no se utiliza también un límite (LIMIT). En cuanto al comportamiento lógico, UNION elimina las filas duplicadas -al actuar de forma equivalente a una operación DISTINCT- mientras que UNION ALL conserva todas las filas y resulta más eficiente, al no requerir comparación entre registros repetidos.

5.2.4. Combinación entre tablas (JOIN)

La sentencia **JOIN** (unir, combinar) de SQL permite combinar registros de una o más tablas de una base de datos.

En la mayoría de los SGBD, la palabra JOIN (sin prefijo) equivale a INNER JOIN siempre que se acompañe de una cláusula ON o USING. Si se omite la condición de unión, el resultado será un producto cartesiano (CROSS JOIN), no una combinación interna.



Atención

El estándar ANSI SQL especifica cinco tipos de JOIN:

- INNER
- LEFT OUTER
- RIGHT OUTER
- FULL OUTER
- CROSS

Matemáticamente, JOIN es composición relacional, la operación fundamental en el álgebra relacional, y, generalizando, es una función de composición.

Sintaxis:

```
SELECT * FROM tabla1 JOIN tabla2 ON tabla1.columna1 = tabla2.columna1;
```



En SQL se distinguen tres grandes categorías de combinaciones:

- Combinación **interna** INNER JOIN.
- Combinación **externa** OUTER JOIN (que puede ser LEFT, RIGHT o FULL)
- Combinación **cruzada** CROSS JOIN.

Indicamos un resumen en la siguiente tabla, antes de estudiarlo con detenimiento:

TIPO JOIN	Tabla izquierda	Tabla derecha
INNER JOIN	Las que cumplen la condición	Las que cumplen la condición
LEFT JOIN	Todas	Las que cumplen la condición
RIGHT JOIN	Las que cumplen la condición	Todas
FULL JOIN	Todas	Todas
CROSS	Presenta el producto cartesiano de los registros de las dos tablas	

Vamos a ver cada uno de ellos con ejemplos, para ello, además de nuestra tabla "jugadores", añadimos una nueva tabla "sueldo" que contiene los salarios base por posición. En esta nueva tabla omitimos la posición centrocampista y añadimos una nueva posición (entrenador) que no está en la tabla jugadores, para así, poder ver claramente las diferencias entre los distintos tipos de JOIN.

posicion	sueldoBase
defensa	2000
delantero	5000
entrenador	6000
portero	1500

Diferencias entre SGDB

El comportamiento de las combinaciones JOIN es prácticamente uniforme en los principales sistemas gestores (Oracle, SQL Server, PostgreSQL, MySQL/MariaDB y SQLite), aunque existen algunas particularidades:

En MySQL/MariaDB, el estándar FULL OUTER JOIN no está implementado directamente; para obtener el mismo resultado se debe simular mediante la unión de un LEFT JOIN y un RIGHT JOIN con UNION. Aunque el estándar ANSI SQL define INNER, LEFT, RIGHT, FULL y CROSS JOIN, no todos los SGDB implementan todos ellos de forma nativa.



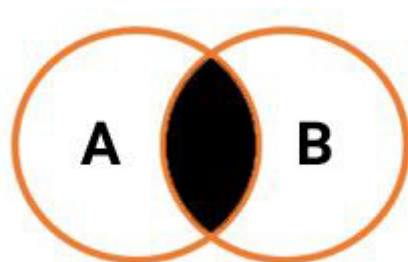
En SQLite, también carece de soporte nativo para RIGHT JOIN y FULL JOIN, aunque puede lograrse el mismo efecto con consultas anidadas o uniones manuales.

Por el contrario, Oracle, PostgreSQL y SQL Server implementan los cinco tipos estándar de JOIN (INNER, LEFT, RIGHT, FULL y CROSS).

En cuanto a sintaxis, en Oracle aún se admite la notación tradicional de join (+) por compatibilidad con versiones antiguas, aunque se recomienda la forma ANSI moderna (JOIN ... ON).

5.2.4.1. Combinación interna INNER JOIN

Combina las filas de una tabla con las de otra tabla, devolviendo únicamente aquellas combinaciones que cumplen la condición especificada en la cláusula ON.



INNER JOIN

Ejemplo INNER JOIN:

```
SELECT jugadores.*, sueldo.sueldoBase FROM jugadores INNER JOIN sueldo ON
jugadores.posicion = sueldo.posicion;
```

Mostrará los registros de las posiciones que aparecen en las 2 tablas.

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	posicion	sueldoBase
1	Iker	Casillas	Fernández	37	portero	San Iker	defensa	2000
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo	delantero	5000
3	Sergi	Barjuan	Esclusa	46	defensa	Corre caminos	entrenador	6000



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	posicion	sueldoBase
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas	portero	1500
5	Carles	Puyol	Saforcada	40	defensa	Tiburón		
6	Zinédine	Yazid	Zidane	46	Centro campista	Zizou		
7	Andrés	Iniesta	Luján	34	Centro campista	Gusiluz		
8	Carlos	Alonso	González	66	delantero	Santillana		
9	Julio	Salinas	Fernández	55	delantero			

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	sueldoBase
1	Iker	Casillas	Fernández	37	portero	San Iker	1500
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo	1500
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos	2000
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas	2000
5	Carles	Puyol	Saforcada	40	defensa	Tiburón	2000
8	Carlos	Alonso	González	66	delantero	Santillana	5000
9	Julio	Salinas	Fernández	55	delantero		5000

Diferencias entre SGBD

El funcionamiento de INNER JOIN es idéntico en todos los principales sistemas gestores de bases de datos (Oracle, SQL Server, PostgreSQL, MySQL/MariaDB y SQLite).

Las diferencias solo aparecen en aspectos sintácticos menores:

- En Oracle, todavía se admite la notación antigua (+) para expresar un outer join, pero no para los inner join, que deben escribirse con la sintaxis ANSI (INNER JOIN ... ON ...).
- En MySQL/MariaDB, la palabra clave INNER es opcional; JOIN por sí sola se interpreta como INNER JOIN.
- En PostgreSQL, SQL Server y SQLite, la forma JOIN también equivale a INNER JOIN por defecto.

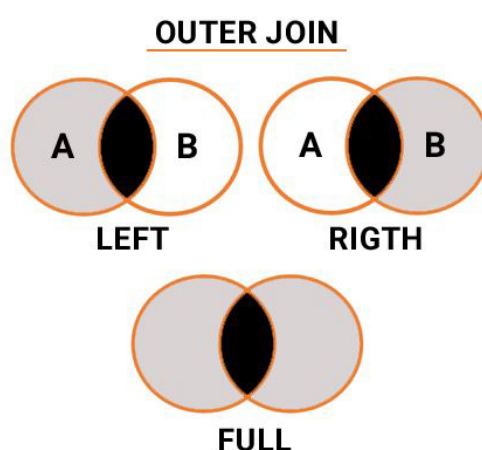


5.2.4.2. Combinación externa OUTER JOIN

La combinación externa (OUTER JOIN) permite conservar los registros de una tabla aunque no tengan correspondencia en la otra.

Como se vio anteriormente, existen tres tipos: LEFT, RIGHT y FULL, que difieren según la tabla cuyos registros se mantienen. En los diagramas de Venn, las áreas en negro representan las coincidencias y las áreas en gris los registros que se conservan sin correspondencia.

A continuación veremos cada uno de ellos con ejemplos prácticos.



Se toman los valores en gris y en negro

5.2.4.2.1. LEFT JOIN

Combina todos los valores de la primera tabla con los valores de la segunda tabla que cumplan una determinada condición, pero también se muestran todos los valores de la primera tabla (LEFT), aunque no cumplan la condición.

Ejemplo LEFT JOIN:

```
SELECT jugadores.*, sueldo.sueldoBase FROM jugadores LEFT JOIN sueldo ON
jugadores.posicion = sueldo.posicion;
```

Esta vez, además de los coincidentes, como en el ejemplo anterior INNER JOIN, se han añadido también los dos registros de **centrocampista** de la primera tabla (jugadores), a pesar de no tener relación en la segunda tabla (sueldos).

En las posiciones que no existen en la tabla sueldo, los valores de la columna sueldoBase aparecen como NULL, indicando que no existe coincidencia en la segunda tabla.



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	posicion	sueldoBase
1	Iker	Casillas	Fernández	37	portero	San Iker	defensa	2000
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo	delantero	5000
3	Sergi	Barjuan	Esclusa	46	defensa	Corre caminos	entrenador	6000
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas	portero	1500
5	Carles	Puyol	Saforcada	40	defensa	Tiburón		
6	Zinédine	Yazid	Zidane	46	Centro campista	Zizou		
7	Andrés	Iniesta	Luján	34	Centro campista	Gusiluz		
8	Carlos	Alonso	González	66	delantero	Santillana		
9	Julio	Salinas	Fernández	55	delantero			

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	SueldoBase
1	Iker	Casillas	Fernández	37	portero	San Iker	1500
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo	1500
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos	2000
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas	2000
5	Carles	Puyol	Saforcada	40	defensa	Tiburón	2000
6	Zinédine	Yazid	Zidane	46	Centro campista	Zizou	
7	Andrés	Iniesta	Luján	34	Centro campista	Gusiluz	



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	SueldoBase
8	Carlos	Alonso	González	66	delantero	Santillana	5000
9	Julio	Salinas	Fernández	55	delantero		5000

Las diferencias entre SGBD en el caso de LEFT JOIN son las mismas que en INNER JOIN, ya explicadas en la introducción general sobre los tipos de JOIN.

5.2.4.2.2. RIGHT JOIN (o RIGHT OUTER JOIN)

Combina los valores de la primera tabla que cumplan una determinada condición con todos los valores de la segunda tabla, pero también se muestran todos los valores de la segunda tabla (RIGHT), aunque no cumplan la condición.

Ejemplo RIGHT JOIN:

```
SELECT jugadores.*, sueldo.sueldoBase FROM jugadores RIGHT JOIN sueldo ON
jugadores.posicion = sueldo.posicion;
```

Ahora además de los coincidentes, como en el ejemplo INNER JOIN, se han añadido también el registro de la segunda tabla sueldos (RIGHT) a pesar de no tener relación en la primera tabla (jugadores). En ese registro, las columnas correspondientes a la tabla jugadores aparecen con valor NULL, ya que no existe coincidencia.

idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	posicion	sueldoBase
1	Iker	Casillas	Fernández	37	portero	San Iker	defensa	2000
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo	delantero	5000
3	Sergi	Barjuan	Esclusa	46	defensa	Corre caminos	entrenador	6000
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas	portero	1500
5	Carles	Puyol	Saforcada	40	defensa	Tiburón		
6	Zinédine	Yazid	Zidane	46	Centro campista	Zizou		



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	posicion	sueldoBase
7	Andrés	Iniesta	Luján	34	Centro campista	Gusiluz		
8	Carlos	Alonso	González	66	delantero	Santillana		
9	Julio	Salinas	Fernández	55	delantero			

Resultado:

idJugador	nombre	apellido1	apellido2	edad	posición	apodo	SueldoBase
5	Carles	Puyol	Saforcada	40	defensa	Tiburón	2000
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas	2000
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos	2000
9	Julio	Salinas	Fernández	55	delantero		5000
8	Carlos	Alonso	González	66	delantero	Santillana	5000
							6000
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo	1500
1	Iker	Casillas	Fernández	37	portero	San Iker	1500



+ Info

En SQLite, RIGHT JOIN no está soportado directamente; puede simularse invirtiendo el orden de las tablas y usando un LEFT JOIN.

5.2.4.2.3. FULL JOIN

Combinación completa: devuelve todas las filas de la primera tabla y todas las de la segunda, emparejándolas cuando existe coincidencia y rellenando con valores NULL cuando no la hay. De este modo, el conjunto resultante contiene todas las filas de ambas tablas: las coincidentes (una sola fila por clave) y las no coincidentes (con NULL en los campos de la tabla donde falta correspondencia).



Ejemplo FULL JOIN:

```
SELECT jugadores.*, sueldo.sueldoBase FROM jugadores FULL JOIN sueldo ON  
jugadores.posicion = sueldo.posicion;
```

Con los datos del ejemplo, un FULL JOIN real devolvería 10 filas:

- 7 posiciones coincidentes (una por cada emparejamiento).
- 2 filas solo presentes en jugadores (los dos centrocampistas, con sueldoBase = NULL).
- y 1 fila solo presente en sueldo (entrenador, con columnas de jugadores = NULL).

No hay filas duplicadas porque cada posición se representa una sola vez.



Aviso

No todos los SGBD implementan FULL OUTER JOIN.

En particular, Microsoft Access, MySQL/MariaDB y SQLite no lo soportan nativamente.

En estos motores se recurre a una simulación combinando resultados de izquierda y de derecha.

A continuación mostramos la consulta equivalente, que generaría duplicados, eso sí.

```
SELECT jugadores.*, sueldo.sueldoBase FROM jugadores  
LEFT JOIN sueldo ON jugadores.posicion = sueldo.posicion  
UNION ALL  
SELECT jugadores.*, sueldo.sueldoBase FROM jugadores  
RIGHT JOIN sueldo ON jugadores.posicion = sueldo.posicion;
```



idJugador	nombre	apellido1	apellido2	edad	posicion	apodo	sueldoBase
1	Iker	Casillas	Fernández	37	portero	San Iker	1500
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo	1500
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos	2000
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas	2000
5	Carles	Puyol	Saforcada	40	defensa	Tiburón	2000
6	Zinédine	Yazid	Zidane	46	Centro campista	Zizou	
7	Andrés	Iniesta	Luján	34	Centro campista	Gusiluz	
8	Carlos	Alonso	González	66	delantero	Santillana	5000
9	Julio	Salinas	Fernández	55	delantero		5000
5	Carles	Puyol	Saforcada	40	defensa	Tiburón	2000
4	Sergio	Ramos	García	32	defensa	Tarzán de Camas	2000
3	Sergi	Barjuan	Esclusa	46	defensa	Correcaminos	2000
9	Julio	Salinas	Fernández	55	delantero		5000
8	Carlos	Alonso	González	66	delantero	Santillana	5000
							6000
2	Luis Miguel	Arconada	Echarri	64	portero	Pulpo	1500
1	Iker	Casillas	Fernández	37	portero	San Iker	1500

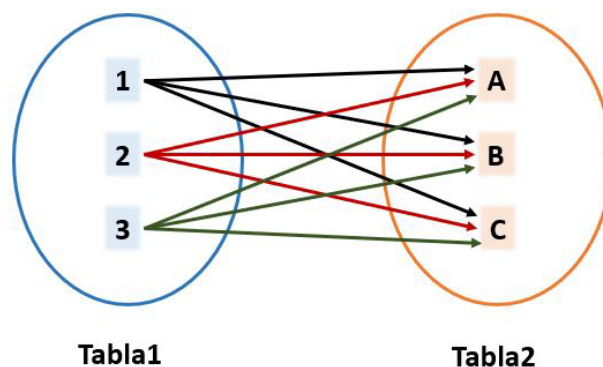
Devolverá 17 filas, porque al usar UNION ALL se suman las del LEFT JOIN (9 filas) a las del RIGHT JOIN (8 filas). Las que coinciden por posición aparecerán duplicadas; además saldrá una fila extra para sueldo.posicion = 'entrenador' sin jugador.



5.2.4.3. Combinación cruzada CROSS JOIN

Presenta el producto cartesiano de los registros de las dos tablas. La tabla resultante tendrá todos los registros de la tabla izquierda combinados con cada uno de los registros de la tabla derecha.

El código SQL para realizar este producto cartesiano enuncia las tablas que serán combinadas, pero no incluye algún predicado que filtre el resultado.



5.3. Consultas de acción

Se denominan así, ya que realizan algún tipo de modificación (acción) sobre los datos, pero no devuelven ningún registro.

Se utilizan para actualizar el contenido de las tablas, (en una sola operación previamente diseñada) mediante acciones como añadir, borrar y modificar registros.

Comandos:

- **INSERT**

Insertar una nueva fila (cargar lotes de datos) en una tabla base de datos en una sola operación.

- **UPDATE**

Modificar valores existentes por otros que especifiquemos.

- **DELETE**

Borrar uno o más registros existentes en una tabla.

También existe **MERGE**, que es considerado por algunos como una sentencia y otros como una cláusula.



5.3.1. INSERT

Agrega uno o más registros (filas) a una (y sólo una) tabla en una sola operación, teniendo en cuenta lo siguiente:

- Las cantidades de columnas y valores deben ser iguales.
- Los valores se especifican en el orden en que se encuentran en la tabla.
- Si un valor no se especifica, le será asignado el valor por defecto.
- Los valores especificados (o implícitos) por la sentencia INSERT deberán satisfacer todas las restricciones aplicables.
- Si ocurre un error de sintaxis o no se cumple alguna restricción, no se agrega la fila y se devuelve un error.

Sintaxis:

```
INSERT INTO nombreTabla VALUES (valor1, valor2..., valorn);
```

Ejemplo:

```
INSERT INTO jugadores VALUES (1, "Iker", "Casillas", "Fernández", 37);
```

idjugador	nombre	apellido1	apellido2	edad
1	Iker	Casillas	Fernández	37
*				

5.3.2. UPDATE

La sentencia UPDATE se utiliza para modificar valores de los atributos, que se especifiquen, en una tabla.



Sintaxis:

```
UPDATE nombre_tabla
SET columna1 = valor1, columna2 = valor2
WHERE columna3 = valor3;
```

Ejemplo: Para el ejemplo vamos a suponer que hemos añadido el atributo posición (como vimos en un punto anterior) y que queremos añadirle un valor.

```
UPDATE jugadores SET posicion = "portero" WHERE idJugador = 1;
```

jugadores					
de Texto Enriquecido, panelContenedorActividad		apellido1	apellido2	edad	posicion
1	Iker	Casillas	Fernández	37	portero
*					

5.3.3. DELETE

Nos permite eliminar una o más tuplas (filas o registros) de una tabla.

Sintaxis:

```
DELETE FROM nombreTabla WHERE nombreColumna = valor;
```

Ejemplo: Borrar todos los registros:

```
DELETE FROM jugadores;
```

Ejemplo: Borrar los que cumplan una determinada condición:

```
DELETE FROM jugadores WHERE nombre = 'Iker';
```



+ Info

Hemos utilizado "WHERE" para indicar la condición de los elementos que hay que borrar. Veremos su funcionamiento más adelante.

5.3.4. MERGE

El comando MERGE permite ejecutar operaciones combinadas de INSERT, UPDATE y DELETE sobre una tabla de destino, a partir de una comparación con una tabla de origen. Es útil para sincronizar datos, especialmente en situaciones donde se desea actualizar si existe coincidencia o insertar si no.

Está disponible en SQL Server, Oracle, PostgreSQL (desde la versión 15) e IBM Db2, con sintaxis basada en el estándar SQL. No permite actualizar una misma fila más de una vez en la misma ejecución, por lo que se considera una operación determinista.

En otros SGBD se emplean instrucciones equivalentes:

- **MySQL y MariaDB:** INSERT ... ON DUPLICATE KEY UPDATE o REPLACE.
- **SQLite:** INSERT OR REPLACE o INSERT ON CONFLICT DO UPDATE.

Estas alternativas cubren funciones similares aunque con diferencias de comportamiento y sintaxis.

6. DCL

Siglas del inglés **Data Control Language**, (en español LDC, siglas de Lenguaje de Control de Datos).

Controla la seguridad de los datos, define los permisos sobre los datos.

Contiene instrucciones mediante las cuales se otorgan o revocan permisos de acceso a los usuarios sobre los objetos de la base de datos; de esta forma, el administrador gestiona la seguridad.

6.1. Comandos DCL

El comando GRANT asigna permisos, y el comando REVOKE, los elimina.

El creador de una tabla tiene por defecto todos los derechos sobre ella. El administrador de la base de datos también puede otorgar a otros usuarios permisos específicos sobre las tablas o revocárselos.



6.1.1. GRANT

Permite a los administradores de las bases de datos dar permisos a uno o varios usuarios o roles para realizar tareas determinadas.

El comando GRANT asigna permisos a uno o varios usuarios sobre una o más tablas u otros objetos de la base de datos, para que puedan realizar determinadas tareas. Los permisos más utilizados son:

- SELECT: autoriza la selección de datos.
- UPDATE: autoriza la modificación de datos.
- DELETE: autoriza la eliminación de datos.
- INSERT: autoriza la inserción de datos.

Se puede usar la opción ALL, para mantener compatibilidad con versiones anteriores.

La sintaxis es:

```
GRANT privilegio [, privilegio2, ...]
ON objeto
TO usuario [, usuario2, ...]
[WITH GRANT OPTION];
```

También se puede usar la opción ALL [PRIVILEGES] para asignar todos los permisos disponibles sobre un objeto.

En SQL Server, un asegurable (del inglés securable) es cualquier objeto sobre el que se pueden conceder permisos, como bases de datos, tablas, vistas, funciones o procedimientos.

Las siguientes asignaciones corresponden a SQL Server:

Si el asegurable es una base de datos, ALL asigna:

- BACKUP DATABASE, BACKUP LOG, CREATE DATABASE, CREATE DEFAULT, CREATE FUNCTION, CREATE PROCEDURE, CREATE RULE, CREATE TABLE, CREATE VIEW.

Si el asegurable es una función escalar, ALL asigna:

- EXECUTE, REFERENCES.



Si el asegurable es una función que retorna una tabla, ALL asigna:

- DELETE, INSERT, REFERENCES, SELECT, UPDATE.

Si el asegurable es un procedimiento almacenado, ALL asigna:

- EXECUTE.

Si el asegurable es una tabla o vista, ALL asigna:

- DELETE, INSERT, REFERENCES, SELECT, UPDATE.

Ordenamos esta información en las siguientes tablas:

Si el asegurable es base de datos, ALL asigna:

BACKUP DATABASE	BACKUP LOG	CREATE DATABASE	CREATE DEFAULT	CREATE FUNCTION	CREATE PROCEDURE	CREATE RULE	CREATE TABLE	BACKUP DATABASE
-----------------	------------	-----------------	----------------	-----------------	------------------	-------------	--------------	-----------------

ALL asigna, si el asegurable es:

	procedimiento almacenado	función escalar	función que retorna una tabla	tabla	Vista
EXECUTE	X	X			
REFERENCES		X	X	X	X
DELETE			X	X	X
INSERT			X	X	X
SELECT			X	X	X
UPDATE			X	X	X

6.1.2. REVOKE

Permite eliminar permisos que previamente se han concedido con GRANT.

Se utiliza para revocar privilegios de acceso a usuarios o roles sobre determinados objetos de la base de datos.



```
REVOKE privilegio [, privilegio2, ...]  
ON objeto  
FROM usuario [, usuario2, ...];
```

Las tareas sobre las que se pueden conceder o denegar permisos son las siguientes:

- CONNECT
- SELECT
- INSERT
- UPDATE
- DELETE
- USAGE

7. Otras subclasificaciones de SQL

Además de los sublenguajes DDL, DML y DCL indicados en ANSI SQL, existen otras subclasificaciones en función de lo que realizan sobre la base de datos.

- **TCL (Transaction Control Language)**: controla las transacciones, permitiendo confirmar o deshacer cambios mediante comandos como COMMIT, ROLLBACK y SAVEPOINT.
- **CCL (Cursor Control Language)**: gestiona el control de cursores, permitiendo recorrer conjuntos de resultados fila a fila para su procesamiento.

7.1. TCL

Permite manejar (controlar el procesamiento de) transacciones en una base de datos relacional mediante unos determinados comandos.

Una transacción es un conjunto de operaciones que se tratan como una única unidad de ejecución.

La transacción solo finaliza con éxito si todas las operaciones que la componen se completan correctamente; por tanto, solo puede tener dos resultados posibles: éxito o fracaso.

Si una de las operaciones no puede ejecutarse, se termina la transacción y se cancelan todos los cambios realizados por las operaciones anteriores.



La reversión de la transacción consiste en devolver la base de datos a su estado inicial antes de que comenzara la ejecución.



Importante

Al iniciar una transacción, los datos de la adición, eliminación y modificación de la declaración SQL, se almacenarán en el registro de administración de transacciones, y no pasarán a almacenarse en la base de datos, hasta que la transacción termine con éxito.

Por defecto, muchos SGBD (como Oracle o PostgreSQL) trabajan en modo autocommit, donde cada sentencia se considera una transacción independiente, a menos que se indique explícitamente lo contrario.

Las transacciones deben cumplir las 4 propiedades ACID:

- **Atomicity** (Atomicidad): dentro de una transacción hay un todo que es indivisible.
- **Consistency** (Consistencia, visto también como Integridad): propiedad que asegura que sólo se empieza aquello que se puede acabar, o bien todos los datos se actualizan (si la transacción es éxito) o todos los cambios se destruyen (si la transacción es fracaso), por tanto, los datos permanecen coherentes antes y después de que se ejecute la transacción.
- **Isolation** (Aislamiento): una transacción en curso no debe afectar ni verse afectada por otras transacciones concurrentes hasta que finalice.
- **Durability** (Durabilidad, visto también como Permanencia): cuando los datos de la gestión de transacciones se envían por completo a la base de datos, los datos se guardarán de forma permanente y serán efectivos. Si se revierte, todos los datos no serán válidos.

Veamos los comandos utilizados para transacciones:

- **COMMIT**
- **ROLLBACK**
- **SAVEPOINT y RELEASE SAVEPOINT**

Se pueden crear puntos de restauración para revertir una transacción, y también eliminar esos puntos de restauración creados.

- **SET TRANSACTION**



7.1.1. COMMIT

Confirma una transacción, guardando de forma permanente todos los cambios realizados en la base de datos, y haciéndolos visibles para otros usuarios y sesiones.

7.1.2. ROLLBACK

En caso de que ocurra algún error en la transacción, la revierte, por tanto, restaura la base de datos al estado original antes de iniciar la transacción.

El comando ROLLBACK, devuelve a la base de datos a algún estado previo, puede ser restaurada a una copia limpia incluso después de que se han realizado operaciones erróneas. (Deshace una transacción).



Recuerda

El comando ROLLBACK se relaciona directamente con el control de INTEGRIDAD en SQL.

Las reversiones son importantes para la integridad de la base de datos.

Son cruciales para la recuperación ante errores de un servidor de base de datos, como por ejemplo un cuelgue del equipo. Al realizar una reversión, cualquier transacción que estuviera activa en el tiempo del cuelgue es revertida y la base de datos se ve restaurada a un estado consistente.

ROLLBACK es el comando que causa que todos los cambios de datos desde la última sentencia BEGIN WORK (prácticamente obsoleto), BEGIN TRANSACTION o START TRANSACTION sean descartados por el sistema de gestión de base de datos relacional (RDBMS), para que el estado de los datos, sea revertido, a la forma en que estaban antes de que aquellos cambios tuvieran lugar.

Una sentencia ROLLBACK también eliminará cualquier punto de recuperación existente que pudiera estar en uso.

En muchos dialectos de SQL, los ROLLBACK son específicos de la conexión. Esto significa que, si se hicieron dos conexiones a la misma base de datos, un ROLLBACK hecho sobre una conexión no afectará a las demás conexiones. Esto es vital para el buen funcionamiento de la concurrencia en la base de datos.



7.1.3. SAVEPOINT

Establece un punto de guardado dentro de una transacción.

Identifica un punto en una transacción a la que más tarde se puede volver.

SAVEPOINT crea puntos de recuperación, dentro de los grupos de transacciones en los que se puede hacer ROLLBACK.

Un SAVEPOINT es un punto de una transacción en el que puede revertir la transacción a un cierto punto sin revertir la transacción completa.

Se crea un punto de recuperación con:

```
SAVEPOINT nombre; //Siendo nombre el identificador elegido para dicho punto de restauración.
```

Todos los cambios realizados después de que un punto de recuperación haya sido declarado con SAVEPOINT pueden ser deshechos emitiendo la sentencia:

```
ROLLBACK TO SAVEPOINT nombre;
```

Una vez que se ha liberado un SAVEPOINT mediante el comando RELEASE SAVEPOINT, ya no puede usarse ROLLBACK TO SAVEPOINT para revertir los cambios posteriores a ese punto.

Diferencias entre SGBD:

- **Oracle, PostgreSQL, MySQL y SQL Server** soportan SAVEPOINT y ROLLBACK TO SAVEPOINT.
- **SQLite** también los implementa desde versiones recientes, pero internamente solo admite un nivel de anidamiento.
- En **SQL Server**, SAVE TRANSACTION es la forma equivalente al comando SAVEPOINT del estándar.



7.1.4. RELEASE SAVEPOINT

Para eliminar un SAVEPOINT creado se utiliza el comando:

(siendo *nombre* del SAVEPOINT a eliminar).

Al liberar un punto de guardado, éste deja de estar disponible para ser utilizado en un ROLLBACK TO SAVEPOINT.

Diferencias entre SGBD:

- **Oracle, PostgreSQL, MySQL y SQLite** admiten el comando RELEASE SAVEPOINT.
- **En SQL Server, no existe** la instrucción RELEASE SAVEPOINT; los puntos de guardado se liberan automáticamente al confirmar (COMMIT) o revertir (ROLLBACK) la transacción.

7.1.5. SET TRANSACTION

Especifica determinadas características para la transacción, como las opciones de nivel de aislamiento y qué segmento de cancelación utiliza.

Permite definir estos parámetros antes de que comience la ejecución efectiva de las operaciones dentro de una transacción.

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Este comando establece que la transacción se ejecutará con el nivel de aislamiento más estricto, evitando lecturas no repetibles y lecturas sucias.

Diferencias entre SGBD:

- Oracle permite además indicar el modo de lectura (READ ONLY o READ WRITE) y, en versiones antiguas, especificar el segmento de rollback.
- PostgreSQL, MySQL y SQL Server admiten el ajuste del nivel de aislamiento mediante SET TRANSACTION ISOLATION LEVEL.
- En SQLite, el nivel de aislamiento se gestiona automáticamente; no se utiliza SET TRANSACTION.



7.2. CCL (Control de Cursores)

CCL, siglas del inglés **C**ursor **C**ontrol **L**anguage, traducido como Lenguaje de control del cursor o control de cursores.

Las bases de datos relacionales se caracterizan porque se ejecutan las operaciones sobre un conjunto de filas, este conjunto puede contener una sola fila o incluso estar vacío, pero es considerado siempre como un conjunto.

Con un cursor se puede apuntar y recorrer fila a fila un conjunto de resultados obtenido por una consulta.

Un cursor es una estructura de control que se utiliza para procesar los resultados de una consulta de forma secuencial, permitiendo leer el contenido fila a fila.

Un cursor suele denominarse también puntero, y dependiendo del tipo de cursor que se declare, podrá desplazarse por el conjunto de resultados y, en algunos casos, modificar o borrar los datos subyacentes.

7.2.1. Creación de un cursor

Cuando declaramos un CURSOR, la instrucción es únicamente para declararlo, especificando las filas o columnas que se van a recuperar, pero la consulta se realizará cuando se abra o se active el cursor.

Sintaxis de declaración de cursor:

```
DECLARE nombreCursor CURSOR FOR especificacionConsulta;
```

especificacionConsulta es la consulta SELECT que define el conjunto de datos sobre el cual operará el cursor.

Por ejemplo:

```
DECLARE c_jugadores CURSOR FOR SELECT columna1, columna2 FROM tabla;
```

En algunos sistemas, como SQL Server, es posible declarar una variable de tipo cursor. En ese caso, se utiliza DECLARE para crear la variable y SET para asignarle el cursor correspondiente.

Si se antepone @ delante del nombre de la variable, significa que es una variable local.



En SQL Server, @ indica una variable local, pero no es exclusivo de cursores, sino de cualquier variable T-SQL.

Sintaxis:

```
DECLARE @variableCursor CURSOR;  
SET @variableCursor = nombreCursor;
```

Diferencias entre SGBD:

- En SQL Server, los cursores pueden asignarse a variables y manipularse como objetos mediante DECLARE y SET.
- En Oracle, los cursores se definen de forma implícita o explícita dentro de bloques PL/SQL.
- En PostgreSQL y MySQL, la declaración de cursores se realiza dentro de procedimientos almacenados o bloques anónimos, sin asignación a variables.
- SQLite no implementa cursores explícitos; su API gestiona automáticamente los resultados fila a fila.

7.2.2. Operaciones

La utilización de un cursor estará compuesta, por una serie de instrucciones, que son:

- **OPEN nombreCursor** (Abrir un cursor)

Como hemos indicado anteriormente, cuando se declara un cursor, únicamente se crea el objeto cursor, pero no se crea el conjunto que va a manipular dicho cursor hasta que se abre (activa), es entonces cuando se ejecuta la consulta y se posiciona el cursor antes de la primera fila, de modo que la primera fila se obtiene mediante la primera instrucción FETCH de esa consulta. Por tanto, esta sentencia no recupera ninguna fila.

- **CLOSE nombreCursor** (Cerrar un cursor)

El puntero que estará sobre un registro desaparece y se liberan los recursos que se están utilizando para mantener el conjunto del cursor, se cierra por tanto, cuando hemos terminado de utilizarlo, porque al cerrarlo ya no podemos recorrer el conjunto de resultados.

- **DEALLOCATE nombreCursor** (Liberar un cursor)

Se elimina la referencia al cursor definido previamente, y ya no es posible realizar una reapertura del mismo.

DEALLOCATE libera definitivamente el cursor y sus recursos; tras ello no puede volver a abrirse.



- **DROP nombreCursor** (Eliminar un cursor)

En la mayoría de los sistemas modernos, como SQL Server o PostgreSQL, no existe el comando DROP CURSOR; la liberación se realiza mediante DEALLOCATE.

DROP CURSOR aparece en algunos lenguajes embebidos o implementaciones antiguas, donde cumple la misma función.

Diferencias entre SGBD:

- En SQL Server, los cursores se gestionan mediante OPEN, FETCH, CLOSE y DEALLOCATE. No existe DROP CURSOR.
- En Oracle, los cursores se cierran automáticamente al salir del bloque PL/SQL, aunque también se puede usar CLOSE.
- En PostgreSQL y MySQL, el ciclo habitual incluye OPEN, FETCH y CLOSE.
- En PostgreSQL los cursores se cierran con CLOSE; DEALLOCATE se utiliza para prepared statements, no para cursores.
- SQLite no dispone de cursores explícitos: el control fila a fila se maneja desde la API del lenguaje host (por ejemplo, C, Python, etc.).

7.2.3. Utilizar un cursor para manipular filas

Un cursor permite **procesar filas individuales de un conjunto de resultados**, normalmente una tabla o una consulta.

Para trabajar con cursores se utilizan, entre otros, los siguientes comandos:

- **DECLARE CURSOR**

Define el cursor y la consulta que determinará el conjunto de resultados sobre el que se operará.

(Este paso se realiza antes de abrir o recorrer el cursor.)

- **FETCH [INTO]**

Recupera la siguiente fila (o una posición concreta, según el tipo de cursor) del conjunto de resultados.

La forma FETCH INTO se emplea en algunos SGBD, como SQL Server o PL/SQL, para almacenar los valores recuperados en variables.

- **UPDATE WHERE CURRENT OF nombreCursor**

Permite modificar la fila actual apuntada por el cursor.



7.2.3.1. FETCH

Este comando recupera la fila actual o una fila determinada del conjunto de resultados.

```
FETCH nombreCursor;
```

Recupera la siguiente fila según el modo de desplazamiento por defecto del cursor.

```
FETCH nombreCursor INTO listaVariables;
```

Con esta sintaxis, en lugar de obtener una fila directamente, almacena los valores obtenidos de las columnas en variables.

Siendo listaVariables una lista de identificadores de variable separados por comas.

Esa lista tiene que tener una variable por cada columna de la sentencia SELECT que define el cursor y los tipos de datos han de ser iguales o compatibles con los tipos de datos de la columna.

Estas opciones solo están disponibles en cursores desplazables (scrollable) y su soporte depende del SGBD.

Se puede recuperar una fila, más rápidamente, de dos formas según su posición:

- Basándose en su **posición absoluta**, con los siguientes comandos:

- **FETCH FIRST**

Recupera la primera fila.

- **FETCH LAST**

Recupera la última fila.

- **FETCH ABSOLUTE n**

Recupera una fila específica desde el principio o desde el final, según el valor de n:

- » Si es valor positivo, desde el comienzo.
- » Si es valor negativo, cuenta desde el final del conjunto de resultados.

El valor de n puede ser una constante o también una variable (@nombrevariable).



- Basándose en su **posición relativa**, con los siguientes comandos:

- **FETCH NEXT**

Recupera la siguiente fila.

- **FETCH PRIOR**

Recupera la fila anterior.

- **FETCH RELATIVE n**

Recupera una fila desde la fila actual según el valor de n:

- » Si es valor positivo: se recupera una fila n **después** de la fila actual.
- » Si es valor negativo: se recupera una fila n **antes** de la fila actual.

El valor de n puede ser una constante o también una variable (@nombrevariable).



Atención

- Si n o @nvar es 0, no se devuelve ninguna fila.
- n debe ser una constante de tipo entero.
- @nvar debe ser de un tipo entero compatible, según el SGBD (por ejemplo INT en SQL Server).

7.2.4. Monitorizar un cursor

Mediante el comando @@FETCH_STATUS se puede monitorizar un cursor para saber cuál fue su resultado según el valor de retorno que nos proporciona este comando, pudiendo ser:

- Valor **0**

El comando FETCH se ejecutó con éxito.

- Valor **-1**

El comando FETCH no se ejecutó con éxito (fallo).



- Valor -2

La fila leída desapareció.

De esta forma se pueden crear estructuras de bucles hasta obtener el resultado deseado.

Diferencias entre SGBD:

- @@FETCH_STATUS es exclusivo de SQL Server.
- En otros SGBD, como Oracle, PostgreSQL o MySQL, el control del estado del cursor se realiza mediante variables de condición, estructuras de control del lenguaje procedural (por ejemplo, EXIT WHEN NOT FOUND en PL/pgSQL) o manejo de excepciones.

8. Procedimientos almacenados

Un procedimiento almacenado (**STORE PROCEDURE**) es un programa precompilado, almacenado físicamente en una base de datos.

Se ejecuta en respuesta a una petición de usuario, directamente en el motor de bases de datos, el cual usualmente corre en un servidor separado.

Los procedimientos se asemejan a las construcciones de otros lenguajes de programación, porque pueden:

- Aceptar parámetros de entrada y devolver varios valores en forma de parámetros de salida al programa que realiza la llamada.
- Contener instrucciones de programación que realicen operaciones en la base de datos.
- Contener llamadas a otros procedimientos.
- Devolver un valor de estado a un programa que realiza una llamada para indicar si la operación se ha realizado correctamente o se han producido errores (indicando el motivo).

Características de los procedimientos almacenados

Tienen las siguientes características:

- Se almacenan en la propia B.D. y constituyen un objeto más dentro de ella.
- Tienden a mejorar el rendimiento de los sistemas.
- Los procedimientos almacenados son reutilizables.

Los usuarios, mediante la aplicación cliente, no necesitan relanzar los comandos individuales, sino que pueden llamar el procedimiento para ejecutarlo en el servidor tantas veces como sea necesario.



Elementos de los procedimientos almacenados

- Parámetros de entrada (pueden recibir parámetros).
- Parámetros de salida (pueden devolver resultados).
- Variables locales (pueden declararse variables en estos procedimientos).
- Cuerpo del procedimiento (acciones que hay que realizar).

Tanto los parámetros de entrada como los de salida son opcionales.

Tipos de procedimientos almacenados

- Procedimientos almacenados del sistema:
 - Son creados por el sistema gestor de bases de datos (SGBD), no por el lenguaje SQL en sí.
 - Nos devuelven información acerca del sistema, sus tablas, contenidos y estructura de los campos, almacenamiento de datos, etcétera.
 - Comienzan con los caracteres sp_ (por ejemplo, sp_tabla). En SQL Server, muchos procedimientos del sistema comienzan por sp_, pero no se recomienda que los usuarios utilicen este prefijo, ya que puede causar conflictos y penalizaciones de rendimiento.
- Procedimientos almacenados definidos por el usuario:
 - Son definidos por el usuario. Tienen cualquier nombre que le dé el usuario. No existe una recomendación estándar.

Diferencias entre SGBD:

- **En SQL Server**, los procedimientos almacenados del sistema comienzan por sp_.
- **En Oracle**, los procedimientos se crean dentro de paquetes PL/SQL y pueden devolver valores mediante parámetros OUT.
- **En MySQL**, la sintaxis de creación (CREATE PROCEDURE) y los delimitadores (BEGIN...END) difieren ligeramente, pero el concepto y las capacidades son equivalentes.
- **En PostgreSQL** históricamente la lógica se implementaba mediante funciones (CREATE FUNCTION); CREATE PROCEDURE existe desde la versión 11 y no devuelve valores directamente, solo mediante parámetros OUT.
- **SQLite** no implementa procedimientos almacenados como tales, pero permite lógica similar mediante extensiones o funciones definidas por el usuario.



8.1. Palabras clave

A continuación, te mostramos las palabras reservadas utilizadas en los procedimientos almacenados:

- **IN:** nos indica que el parámetro será de entrada.
- **OUT** u **OUTPUT:** nos indica que el parámetro será de salida.
- **INOUT:** nos indica que el parámetro será de entrada y salida.
- **AS:** introduce el cuerpo del procedimiento en algunos SGBD (por ejemplo, SQL Server); en otros se usa IS/AS (Oracle) o no se utiliza así (MySQL).
- **BEGIN:** limitador que indica el comienzo del cuerpo del procedimiento.
- **END:** limitador que indica el final del cuerpo del procedimiento.
- **DELIMITER:** (en clientes de MySQL/MariaDB) cambia temporalmente el carácter que finaliza la sentencia para que el propio punto y coma que aparece dentro del cuerpo del procedimiento no corte la instrucción CREATE PROCEDURE. Una vez creado el procedimiento se vuelve a poner ';' con DELIMITER ; .
- **CALL:** realiza una llamada a un procedimiento existente.

Sentencias básicas

Vamos a ver cuatro sentencias básicas: CREATE PROCEDURE, EXECUTE PROCEDURE, ALTER PROCEDURE, DROP PROCEDURE.

- **CREATE PROCEDURE:** se utiliza para crear el procedimiento.

Sintaxis:

```
CREATE PROCEDURE nombre_procedimiento [lista_parametros] AS  
Sentencias_del_procedimiento
```

- **EXECUTE (o CALL):** sirve para invocar un procedimiento almacenado. Los parámetros son opcionales.

Sintaxis:

```
EXEC / EXECUTE nombre_procedimiento [lista_parametros] (SQL Server)  
CALL nombre_procedimiento([lista_parametros]); (estándar / MySQL / PostgreSQL)
```



- **ALTER PROCEDURE:** lo usamos para modificar un procedimiento almacenado.

Sintaxis:

```
ALTER PROCEDURE nombre_procedimiento [lista_parametros] AS  
sentencias_del_procedimiento
```

- **DROP PROCEDURE:** con esta sentencia podemos eliminar un procedimiento almacenado.

Sintaxis:

```
DROP PROCEDURE nombre_procedimiento;
```

Especificación de los parámetros

En SQL Server los parámetros formales de un procedimiento NO llevan @; se declaran sin símbolo (nombre tipo). El carácter @ se usa para variables locales DENTRO del cuerpo del procedimiento y para pasar valores cuando se invoca el procedimiento con @nomPar = valor. En MySQL/PostgreSQL los parámetros formales se escriben sin @.

Si un usuario no especifica el valor de un argumento al llamar a un procedimiento, este recibe su valor por defecto.

Sintaxis:

```
@nombre_parametro tipo_dato [= valor_defecto] [OUTPUT] //SQL Server  
nombre_parametro [IN | OUT | INOUT] tipo_dato [DEFAULT valor_defecto]  
//MySQL/PostgreSQL/Oracle
```

Ejemplo

Vamos a crear un procedimiento que inserte un registro en la tabla jugadores si la edad es menor o igual a 40 años y en leyendas si es mayor.



```

/* EJEMPLO COMPLETO -- CÓDIGO SQL SERVER */
GO

CREATE PROCEDURE baseDeJugadores.spu_addJugador
    @nombre VARCHAR(250),
    @apellido1 VARCHAR(250),
    @apellido2 VARCHAR(250),
    @edad INT,
    @posicion VARCHAR(25),
    @apodo VARCHAR(250)
AS
BEGIN
    IF @edad > 40
        INSERT INTO leyendas (nombre, apellido1, apellido2, edad, posicion, apodo)
        VALUES (@nombre, @apellido1, @apellido2, @edad, @posicion, @apodo);
    ELSE
        INSERT INTO jugadores(nombre, apellido1, apellido2, edad, posicion, apodo)
        VALUES (@nombre, @apellido1, @apellido2, @edad, @posicion, @apodo);
END;
GO

/* LLAMADA */
EXEC baseDeJugadores.spu_addJugador
    'Michael', 'Laudrup', '', 54, 'centrocampista', 'Kongen';

-- Esta llamada inserta al jugador en la tabla leyendas.

```

Diferencias entre SGBD:

- **En SQL Server**, los parámetros se indican con @ y se usa EXECUTE o EXEC para ejecutar procedimientos.
- **En MySQL y PostgreSQL**, se utiliza CALL nombre_procedimiento() y la palabra DELIMITER pertenece al cliente, no al lenguaje SQL.
- **En Oracle**, los procedimientos pueden formar parte de paquetes (CREATE OR REPLACE PROCEDURE) y no utilizan el prefijo @ para parámetros.
- **SQLite** no dispone de procedimientos almacenados ni de funciones externas en su distribución oficial; cualquier 'simulación' requiere código externo (por ejemplo, en C o Python).



8.2. Eventos y disparadores (*Triggers*)

A partir de MySQL 5.0.2 se incorporó el soporte básico para disparadores (triggers).

Sin embargo, los disparadores ya existían previamente en otros sistemas de gestión de bases de datos como Oracle, SQL Server o PostgreSQL.

Podemos decir que es un tipo especial de procedimiento almacenado, que el sistema ejecuta de manera automática cuando sucede algún evento sobre las tablas de la base de datos a las que se encuentre asociado.

Vamos a resumir el funcionamiento de un trigger:

- Un *trigger* puede dispararse antes o después de un evento, mediante el modificador BEFORE (antes) o AFTER (después).

Es decir, antes o después de que se ejecute la sentencia SQL que contiene la cláusula DELETE, INSERT o UPDATE.

- La activación del trigger provocará un tipo de operación DML:
 - INSERT
 - DELETE
 - UPDATE
- Se puede indicar una condición que se debe cumplir para que se ejecute el *trigger*, o no tener condición y lanzarse siempre que se produzca el evento.
- Si incluimos el modificador OF, el *trigger* se ejecutará solamente cuando la sentencia SQL afecte a los campos incluidos en la lista.
- Alcance del disparador:
 - Fila:

Los disparadores con nivel de fila, se identifican por la cláusula **FOR EACH ROW** en la definición del disparador, que indica que el trigger se disparará cada vez que se realizan operaciones sobre cada fila de la tabla.

Si se acompaña del modificador **WHEN**, se establece la restricción de que el trigger solo actuará, sobre las filas que satisfagan la restricción.

- Sentencia:

Los disparadores con nivel de sentencia se activan sólo una vez, antes o después de la orden.



Resumiendo

Estructura de un trigger (modelo evento-condición-acción), se especifican tres elementos:

- Evento que lo desencadena (INSERT, DELETE O UPDATE).
- Condición que se debe cumplir para que se ejecute el trigger (puede no tener condición y lanzarse siempre que se produzca el evento).
- Acciones que se realizan si se ejecuta el trigger.

El argumento INSTEAD OF (en vez de)

Se utiliza para indicar que se inicia el trigger en vez de la instrucción SQL que lo debe disparar.

Indicamos características de su utilización:

- No puede especificar INSTEAD OF para los desencadenadores DDL o LOGON.
- Como máximo, puede definir un desencadenador INSTEAD OF por cada instrucción INSERT, UPDATE o DELETE en una tabla o vista. También puede definir otras vistas en las vistas que tengan su propio desencadenador INSTEAD OF.
- No puede definir desencadenadores INSTEAD OF en vistas actualizables que usan WITH CHECK OPTION. Al hacerlo se genera un error cuando se agrega un desencadenador INSTEAD OF a una vista actualizable para la que se ha especificado WITH CHECK OPTION. Puede quitar esta opción mediante ALTER VIEW antes de definir el desencadenador INSTEAD OF.
- Para los desencadenadores INSTEAD OF, no puede utilizar la opción DELETE en tablas que tengan una relación referencial que especifica una acción ON DELETE en cascada.
- Para los desencadenadores INSTEAD OF, no se permite la opción UPDATE en tablas que tengan una relación referencial que especifica una acción ON UPDATE en cascada.



+ Info

Los triggers INSTEAD OF son compatibles con SQL Server, Oracle y PostgreSQL sobre vistas, pero no están disponibles en MySQL.



Diferencias de un trigger con los procedimientos almacenados del sistema:

- Un trigger no puede ser invocado directamente.
- Al intentar modificar los datos de una tabla asociada a un disparador, el disparador se ejecuta automáticamente.
- No reciben ni devuelven parámetros.
- Se utilizan para mantener la integridad de los datos.

Limitaciones en la sentencia que ejecutará el disparador

Existen limitaciones sobre lo que puede aparecer dentro de la sentencia que el disparador ejecutará al activarse:

- En MySQL el trigger no puede hacer INSERT/UPDATE/DELETE sobre la misma tabla que lo dispara; en SQL Server u Oracle sí está permitido
Sí que se pueden emplear las palabras clave OLD y NEW.
 - **OLD:** se refiere a un registro existente que va a borrarse o que va a actualizarse antes de que esto ocurra.
 - **NEW:** se refiere a un registro nuevo que se insertará o a un registro modificado luego de que ocurre la modificación.
- El disparador no puede utilizar sentencias que inicien o finalicen una transacción, tal como START TRANSACTION, COMMIT, o ROLLBACK.

8.2.1. Crear o eliminar un TRIGGER

Los disparadores se pueden crear o eliminar mediante las sentencias:

- **CREATE TRIGGER** crear disparador.
- **DROP TRIGGER** eliminar disparador.



+ Info

En MySQL, a partir de la versión 5.0.2, se incorporó el uso de triggers, y en sus primeras versiones ambas sentencias requerían poseer el privilegio SUPER.



En versiones más recientes (a partir de MySQL 5.7), el privilegio requerido es TRIGGER.

8.2.1.1. Sintaxis de CREATE TRIGGER

Cuando creamos un disparador, este queda asociado a una tabla, que debe ser una tabla permanente, no puede ser una tabla TEMPORARY ni, en el caso de MySQL una vista.

En otros SGBD como SQL Server, Oracle o PostgreSQL se pueden definir triggers sobre vistas mediante INSTEAD OF, que no forman parte del estándar SQL.

La sintaxis (dialecto MYSQL) es:

```
CREATE TRIGGER nombredisp  
Momentodisp  
Eventodisp  
ON nombretabla  
FOR EACH ROW  
sentenciadisp;
```

Donde:

- **nombredisp.**

Es el nombre que le damos al disparador.

- **momentodisp.**

Es el momento en que el disparador entra en acción; se indica si debe ejecutarse antes o después de la sentencia que lo activa: BEFORE (antes) o AFTER (después).

- **eventodisp.**

Indica la clase de sentencia que activa al disparador. En MySQL, pueden ser: INSERT, UPDATE o DELETE.

El disparador BEFORE para sentencias INSERT puede utilizarse para validar los valores a insertar.



Se pueden tener al mismo tiempo, en MySQL, los disparadores:

- BEFORE UPDATE y BEFORE INSERT.
- BEFORE UPDATE y AFTER UPDATE.
- **nombretabla.**

Indica a qué tabla queda asociado el disparador (que debe ser una tabla permanente).

- **sentenciadisp.**

Es la sentencia que se ejecuta cuando se activa el disparador.

Si se desean ejecutar múltiples sentencias, deben colocarse entre BEGIN y END (bloque compuesto), según lo permita el SGBD.

Cada sentencia indicada debe terminar con un punto y coma (;) delimitador de sentencias.

En MySQL:

En la consola de MySQL, puede ser necesario cambiar temporalmente el delimitador con DELIMITER // para evitar que el cliente interprete los puntos y coma internos como el final del comando.

Este cambio no forma parte del lenguaje SQL, sino del propio cliente de MySQL, y permite escribir bloques BEGIN...END con varias sentencias internas.

Al finalizar la creación del trigger, se restablece el delimitador habitual con DELIMITER ;.

A continuación mostramos un ejemplo:

```
DELIMITER //  
  
CREATE TRIGGER ejemplo_trigger BEFORE INSERT ON mi_tabla  
FOR EACH ROW  
BEGIN  
    INSERT INTO tabla_auditoria (accion) VALUES ('insert');  
    SET NEW.fecha_creacion = NOW();  
END//  
DELIMITER ;
```



8.2.1.2. Sintaxis de DROP TRIGGER

Con la sentencia DROP se elimina un disparador.

La sintaxis en dialecto MYSQL es:

```
DROP TRIGGER nom_disp
```

- **Nombredisp.**

Es el nombre del disparador a eliminar.



+ Info

A partir de la versión MySQL 5.0.10, se indica la sintaxis:

```
DROP TRIGGER [nombreesquema.]nombredisp
```

Siendo el nombreesquema opcional, y si se omite, el disparador se elimina en el esquema actual.

En versiones antiguas (anteriores a MySQL 5.0.10), si se realizaba una actualización desde una versión previa, era necesario eliminar todos los disparadores antes de actualizar y volver a crearlos posteriormente.

Esta limitación ya no aplica en versiones modernas de MySQL.

8.2.2. Extensiones OLD y NEW de MySQL para los disparadores

Las palabras clave OLD y NEW (que no son sensibles a mayúsculas) permiten acceder a columnas en los registros afectados por un disparador.

- En un disparador para INSERT, solamente puede utilizarse NEW.nom_col; ya que no hay una versión anterior del registro.



- En un disparador para DELETE sólo puede emplearse OLD.nom_col, porque no hay un nuevo registro.
- En un disparador para UPDATE se puede emplear OLD.nom_col para referirse a las columnas de un registro antes de que sea actualizado, y NEW.nom_col para referirse a las columnas del registro luego de actualizarlo.
- Una columna precedida por OLD es de sólo lectura. Es posible hacer referencia a ella, pero no modificarla
- Una columna precedida por NEW puede ser referenciada si se tiene el privilegio SELECT sobre la tabla.
- En un disparador BEFORE, también es posible cambiar su valor si se tiene el privilegio de UPDATE sobre la tabla, con:

```
SET NEW.nombre_col = valor;
```

Esto significa que, en MySQL, un disparador BEFORE puede usarse para modificar los valores antes de que se inserten en un nuevo registro o se utilicen para actualizar uno existente.

- En un disparador BEFORE, el valor de NEW para una columna AUTO_INCREMENT es 0, porque el número secuencial aún no ha sido generado por el motor de base de datos; se asignará automáticamente cuando el registro sea realmente insertado.

Diferencias entre SGBD:

Aunque las palabras clave OLD y NEW son específicas de MySQL, existen mecanismos equivalentes en otros sistemas:

- **En PostgreSQL**, se utilizan también OLD y NEW.
- **En Oracle**, la sintaxis es la misma dentro de los triggers PL/SQL.
- **En SQL Server**, los valores anteriores y nuevos se manejan mediante las pseudo-tablas deleted y inserted.

8.2.3. Gestión de errores

En MySQL, los errores ocurridos durante la ejecución de disparadores se gestionan de la siguiente forma:

- Si lo que falla es un disparador BEFORE, no se ejecuta la operación en el correspondiente registro.
- Un disparador AFTER se ejecuta solamente si el disparador BEFORE (de existir) y la operación se ejecutaron exitosamente.



- En MySQL, un error durante la ejecución de un disparador BEFORE o AFTER provoca la falla de toda la sentencia que invocó el disparador.
- En tablas transaccionales, la falla de un disparador (y, por lo tanto, de toda la sentencia) debería causar la cancelación (ROLLBACK) de todos los cambios realizados por esa sentencia.

En tablas no transaccionales, cualquier cambio realizado antes del error no se ve afectado.

Diferencias entre SGBD

En MySQL, este comportamiento depende del motor de almacenamiento utilizado:

- Las tablas InnoDB son transaccionales, por lo que ante un error en un disparador se revierte la sentencia que provocó la ejecución del disparador.
- Las tablas MyISAM no son transaccionales, por lo que los cambios anteriores al error permanecen aplicados.

En otros SGBD como PostgreSQL, Oracle o SQL Server, todos los triggers se ejecutan dentro del contexto de una transacción, y cualquier error provoca automáticamente un rollback completo de la operación.

8.3. Snapshots en SQL

Un snapshot (o instantánea) permite crear una representación del estado lógico de una base de datos en un momento determinado, según el mecanismo implementado por el SGBD, posibilitando, entre otras cosas, la generación de informes a partir de los datos contenidos en esa instantánea.

Estas instantáneas permiten consultar, y en algunos SGBD revertir, el estado previo de la base de datos en un instante concreto, lo que resulta útil para revertir manualmente los cambios realizados (como eliminaciones de tablas, modificaciones de procedimientos almacenados u operaciones CRUD), sin necesidad de restaurar una copia de seguridad completa.

Usar snapshots puede ser de gran utilidad en procesos donde sea necesario aplicar cambios en la estructura de la base de datos y se requiera volver rápidamente a un estado anterior o compararlo con el actual.

Los snapshots también pueden emplearse al aplicar cambios críticos por lotes en una base de datos productiva, permitiendo volver a un punto anterior rápidamente, sin tener que restaurar un backup completo, cuyo proceso suele ser más lento.



Atención

Diferencia entre SNAPSHOTS y ROLLBACK:

- **ROLLBACK:** pertenece al sistema de transacciones. Revierten automáticamente una transacción que no se completa correctamente (recuerda las propiedades ACID).
- **SNAPSHOTS:** crean una instantánea del estado lógico de la base de datos, según el SGBD, en un momento concreto, permitiendo volver manualmente a ese punto o consultarlo cuando el SGBD lo permite. No forman parte de las transacciones.

Diferencias entre SGBD

- **En SQL Server**, un snapshot es una base de datos de solo lectura creada con `CREATE DATABASE ... AS SNAPSHOT OF ...`, que puede usarse para revertir o consultar el estado anterior de otra base de datos.
- En otros SGBD, como **Oracle**, **PostgreSQL** o **MySQL**, no existe un comando SNAPSHOT equivalente al de SQL Server para bases de datos completas; los efectos similares se logran mediante copias físicas, backups en caliente o MVCC, pero no permiten revertir una base de datos completa a un punto anterior de forma directa.

9. Bibliografía

- SILBERSCHATZ, K., SUDARSHAN, S. y KORTH, H. Fundamentos de bases de datos 5.ª edición. McGraw-Hill, 2014.
- <http://www.hacienda.gob.es/Documentacion/Publico/Curso/T13.pdf>.
- <https://docs.microsoft.com/es-es/sql/t-sql/language-elements/transactions-transact-sql?view=sql-server-2017>.
- <http://es.wikipedia.org>.
- <http://sql.11sql.com/sql-order-by.htm>.
- <https://msdn.microsoft.com/es-es/vba/access-vba/articles/create-and-delete-tables-and-indexes-using-access-sql>.



- <https://www.campusmvp.es/recursos/post/Fundamentos-de-SQL-Como-realizar-consultas-simples-con-SELECT.aspx>.
- http://www.mundoracle.com/subconsultas.html?Pg=sql_plsql_6.htm.
- <https://docs.microsoft.com/es-es/sql/t-sql/statements/statements?view=sql-server-2017#data-manipulation-language>.
- <https://www.w3schools.com/sql/>.
- <https://elbauldelprogramador.com/plsql-disparadores-o-triggers/>.
- <http://dbadixit.com/transaction-control-language-tcl-del-sql/#:~:text=El%20Lenguaje%20de%20control%20de,aclarar%20el%20concepto%20de%20transacci%C3%B3n>.
- <http://www.tutorialesprogramacionya.com/oracleya/temarios/descripcion.php?cod=261&punto=1&inicio=>.
- <https://docs.microsoft.com/es-es/sql/relational-databases/stored-procedures/stored-procedures-database-engine?view=sql-server-2017>.
- https://www.ibm.com/support/knowledgecenter/es/SSEPGG_8.2.0/com.ibm.db2.udb.dc.doc/dc/c_sp.htm.
- https://www.ecured.cu/Procedimientos_almacenados.
- <https://docs.microsoft.com/es-es/sql/t-sql/statements/create-trigger-transact-sql?view=sql-server-2017>.
- <https://docs.microsoft.com/es-es/sql/relational-databases/triggers/dml-triggers?view=sql-server-2017>.
- <https://manuales.guebs.com/mysql-5.0/triggers.html>.
- <https://lasdiferencias.com/diferencias-sql-dinamico-estatico/>
- <https://www.ibm.com/docs/es/rational-soft-arch/9.6.1?topic=statements-select>
- <https://learn.microsoft.com/es-es/sql/t-sql/data-types/binary-and-varbinary-transact-sql?view=sql-server-ver16>
- <https://learn.microsoft.com/es-es/sql/t-sql/data-types/constants-transact-sql?view=sql-server-ver16>
- <https://www.1keydata.com/es/sql/sql-constraint.php>

